Fundamentos de Aprendizagem Automática

Class 06 — Decision Trees and k-Nearest Neighbours

Mário Antunes

Universidade de Aveiro

2026

Models Without a Formula

Where We Left Off

linear
a line (Class 03)

probabilistic
a distribution
(Class 04)

margin, network
kernel, layers
(Class 05)

Class 06: no fitted formula: *rules* and *neighbours*

- Until now a model was **a formula with parameters** fitted by an optimizer
- Today two **algorithms on the data**: a *tree* of questions and the *nearest neighbours*
- Both are simple to explain, and powerful when used with care

What We Do Today

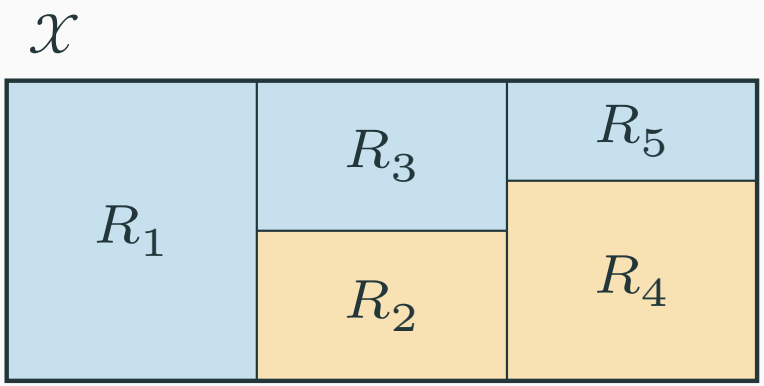
- **Decision tree:** yes/no questions end in a prediction; the *symbolist* of Project 1
- **Nearest neighbours:** the k most similar cases vote; the *analogizer*
- Both are **non-parametric**: the model grows with the *data*, not with a fixed parameter count
- They keep **hyperparameters**: depth, leaf size, k, the distance
- Both are easy to explain, and both fail in instructive ways

Roadmap of the Class

Part	Question	Answer
Models without a formula	what do they share?	regions of the feature space
	what is the price?	data and dimension
Decision trees	which question first?	the purest groups
	why not the best tree?	NP-complete: greedy
	how to use categories?	"value in a subset"
	when to stop?	before it fits the noise
k-nearest neighbours (KNN)	what is "similar"?	a scaled distance
	how many, how fast?	tune k; prune, approximate
Choosing	which learner?	one protocol, a test

Terminology: Cells and Regions

Both models give one answer per *piece* of the feature space.



$A_0 = R_1 \cup R_3 \cup R_5$

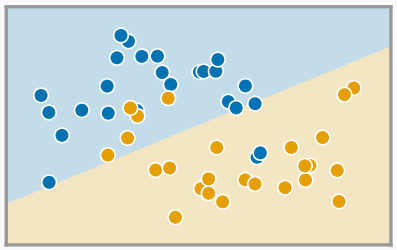
$A_1 = R_2 \cup R_4$

term	meaning	symbol
feature space	all values of the d features	$\mathcal{X}$
cell	a piece of $\mathcal{X}$ with one answer	R_j
answer	the class a cell predicts	$\hat{y}_j$
answering cells	indices of the cells that answer y	J_y
partition	cells that cover $\mathcal{X}$, no overlap	$\{R_j\}$
class region	all cells that answer class y	$A_y = \bigcup_{j \in J_y} R_j$

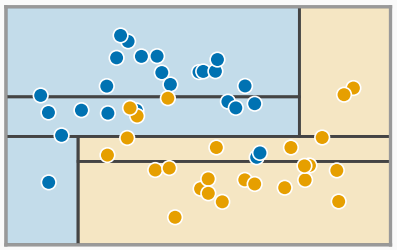
Linear Model and Tree: Half-Planes and Boxes

The same 50 points, two classes: a linear model has two cells, a tree one box per leaf.

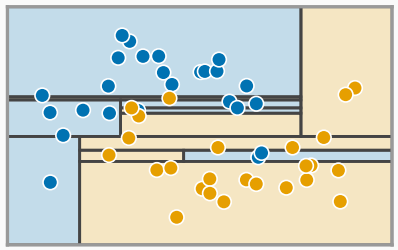
linear
2 half-planes



tree, depth 3
6 boxes



tree, depth 8
13 boxes

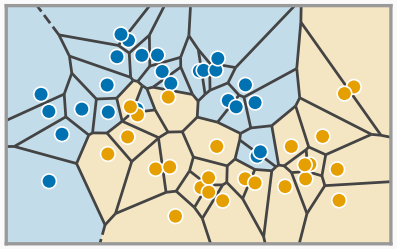


- Linear: one cut, two half-planes; training accuracy 0.840
- **Tree, depth 3:** 6 boxes, 0.900; **depth 8:** 13 boxes, 1.000
- The tree border is a staircase of axis-parallel steps

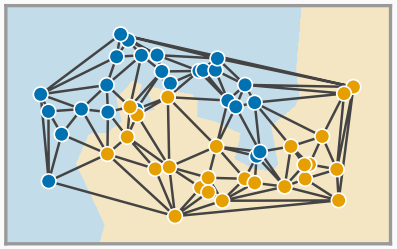
Nearest Neighbours: Voronoi Cells

Each stored case owns a cell: the points nearer to it than to any other case.

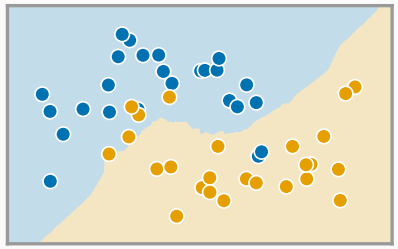
1-NN
50 Voronoi cells



Delaunay triangles
who is next to whom



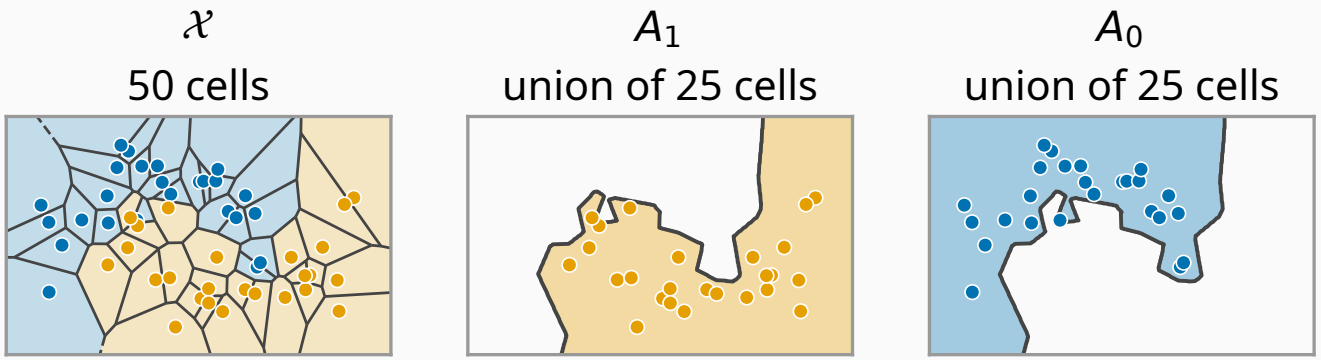
15-NN
smooth border



- The **Delaunay triangulation** joins cells that share a border: 92 triangles
- The decision border follows only 33 of the 141 shared borders
- Fifteen neighbours smooth it: training accuracy 0.860, one neighbour 1.000

A Class Is a Set of Cells

The region of a class is the union of the cells that answer it.

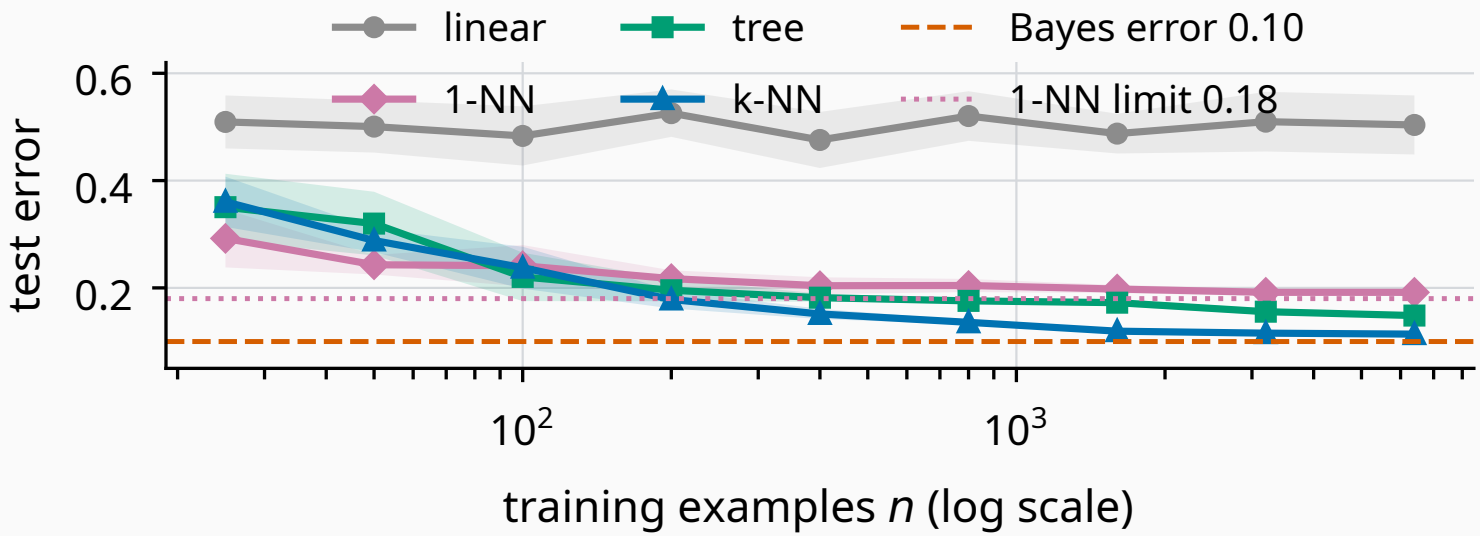


$$A_y = \bigcup_{j \in J_y} R_j \quad \text{(definition of the class region)}$$

- One neighbour, 50 points: A_0 has 25 cells (0.480 of the box), A_1 25 (0.520)
- The two regions overlap nowhere and together cover $\mathcal{X}$
- The field: set theory and computational geometry

Any Region, Given Enough Cells

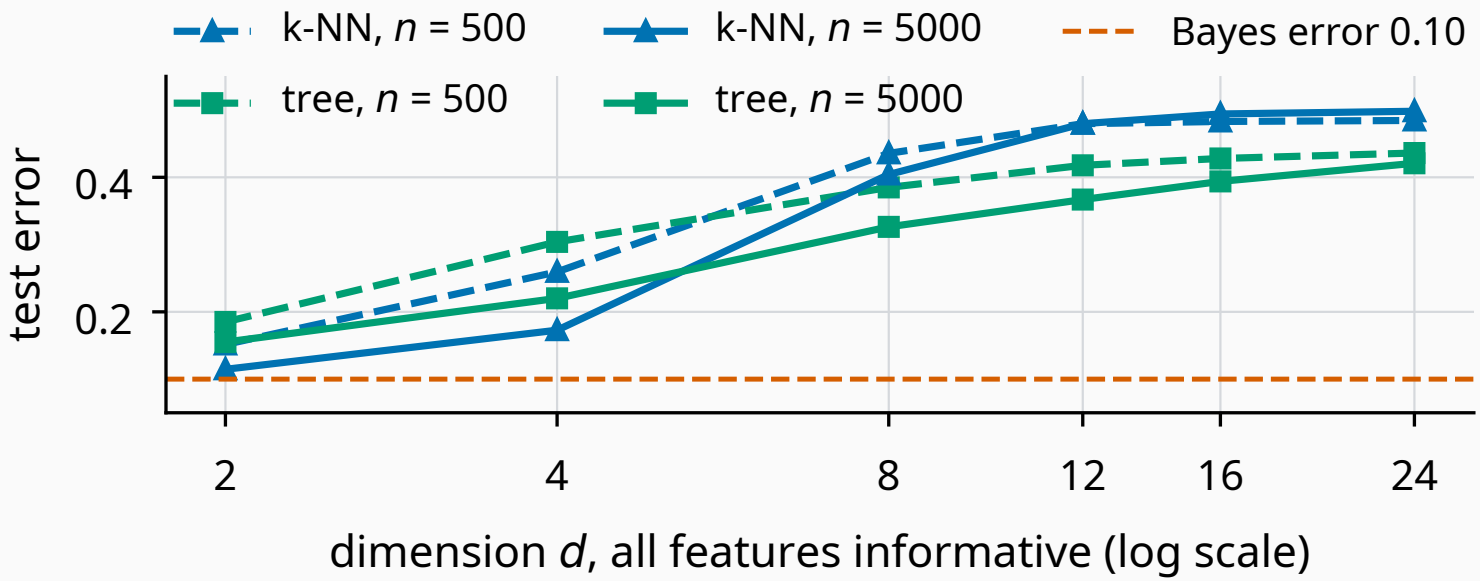
A circle is neither a half-plane nor a box. Test error, 10% of the labels flipped:



- **Linear:** 0.5; tree 0.148; neighbours 0.114 ($n = 6400$)
- One neighbour: 0.192; limit $2 \times 0.10 \times 0.90 = 0.18$ (Cover and Hart 1967)
- Neighbours are consistent (Stone 1977); trees too (Breiman et al. 1984)

The Price: Data and Dimension

Without a functional form the price is data: the cells needed grow exponentially with the dimension d.

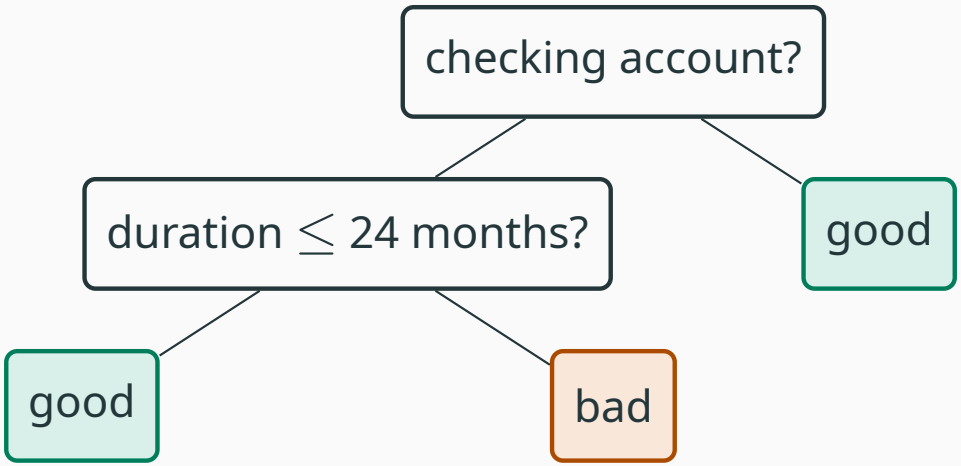


- Cells of side 0.5: 16 for $d = 2$, 17 million for $d = 12$
- At $d = 2, 8, 16$ ($n = 5000$): neighbours 0.115, 0.404, 0.494
- The tree: 0.155, 0.326, 0.394

Decision Trees

A Tree Is a Sequence of Questions

Everyone has used one: a flow chart. A bank deciding on a loan might ask:



- Each **internal node** asks about *one* feature; each **branch** is an answer
- Each **leaf** gives the prediction: a class or a number
- A new case follows one path: at most *depth* questions

A Tree Is a Set of Rules

A depth-3 tree of the lab (426 tumours): each path is an `if ... and ... rule`.

```
if worst_concave_points <= 0.147:
    if worst_radius <= 17.7:
      if worst_perimeter <= 108:  -> benign    (249 cases, 5 malignant)
       else:                        -> benign    (24 cases, 7 malignant)
    else:
      if worst_concavity <= 0.191: -> benign   (5 cases, 2 malignant)
       else:                       -> malignant (16 cases, all malignant)
else: ... -> malignant (131 cases, 2 benign)
```

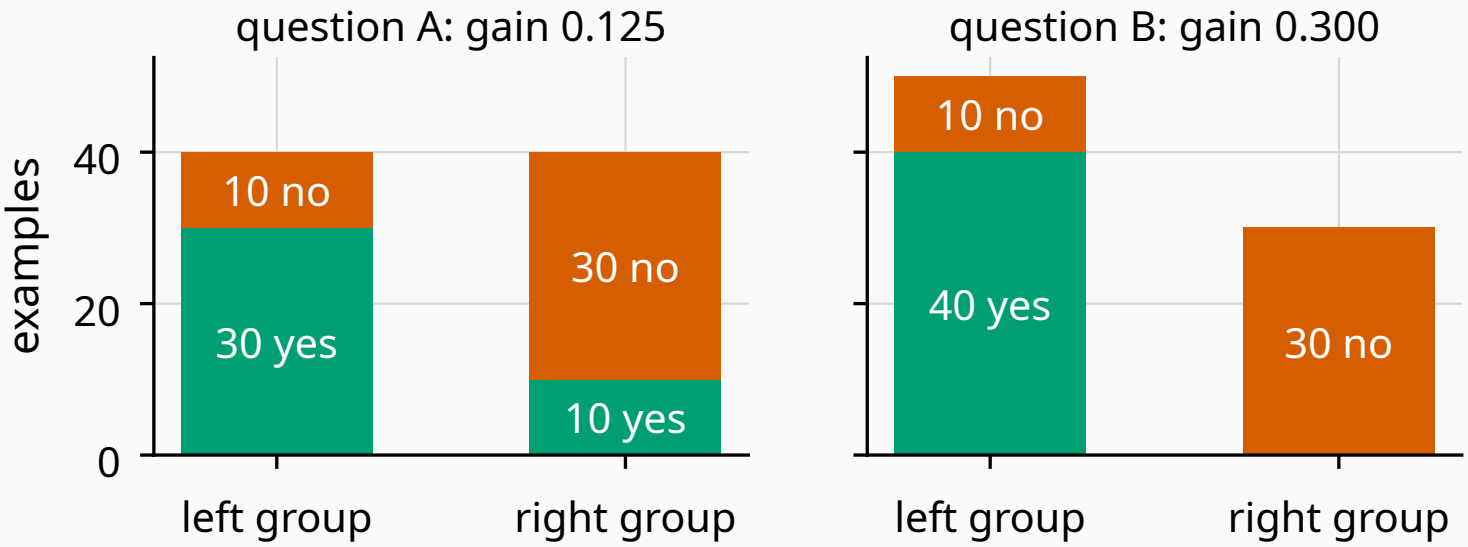
- A human can **read, check and challenge** every rule
- The counts show how *sure* a rule is: one case is one patient

Notation: Questions and Gain

symbol	meaning	used in
n	examples in the group a question splits	gain
n_L	examples that go to the left child	gain
n_R	examples that go to the right child	gain
I_0	impurity of the parent group	gain
I_L	impurity of the left child	gain
I_R	impurity of the right child	gain
g	gain of a question	gain

Which Question First?

A good question makes the groups it creates purer.
Example: 80 examples (40 and 40), two candidate questions.



- We need a **number** for the impurity of a group
- The best question is then the one with the largest **gain**

Measures of Impurity

For a group with class shares $p_1, \ldots, p_K$, three measures:

- **Gini index:** $G = 1 - \sum_k p_k^2$, the chance that two random examples differ
- **Entropy:** $H = -\sum_k p_k \log_2 p_k$, the bits needed to name the class (Class 01)
- **Error:** $E = 1 - \max_k p_k$, the share the majority vote gets wrong
- All are 0 for a pure group and largest for equal classes

The Three Curves

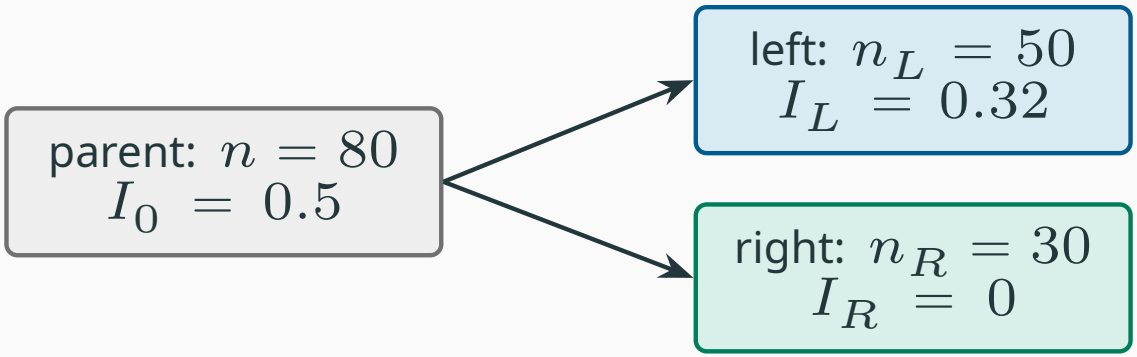
Impurity of a two-class node against the share of class 1:



- Entropy and Gini are smooth: a purer split always lowers them
- The error is a straight line: many useful splits leave it unchanged

The Gain of a Question

A question splits n examples into n_L on the left and n_R on the right.

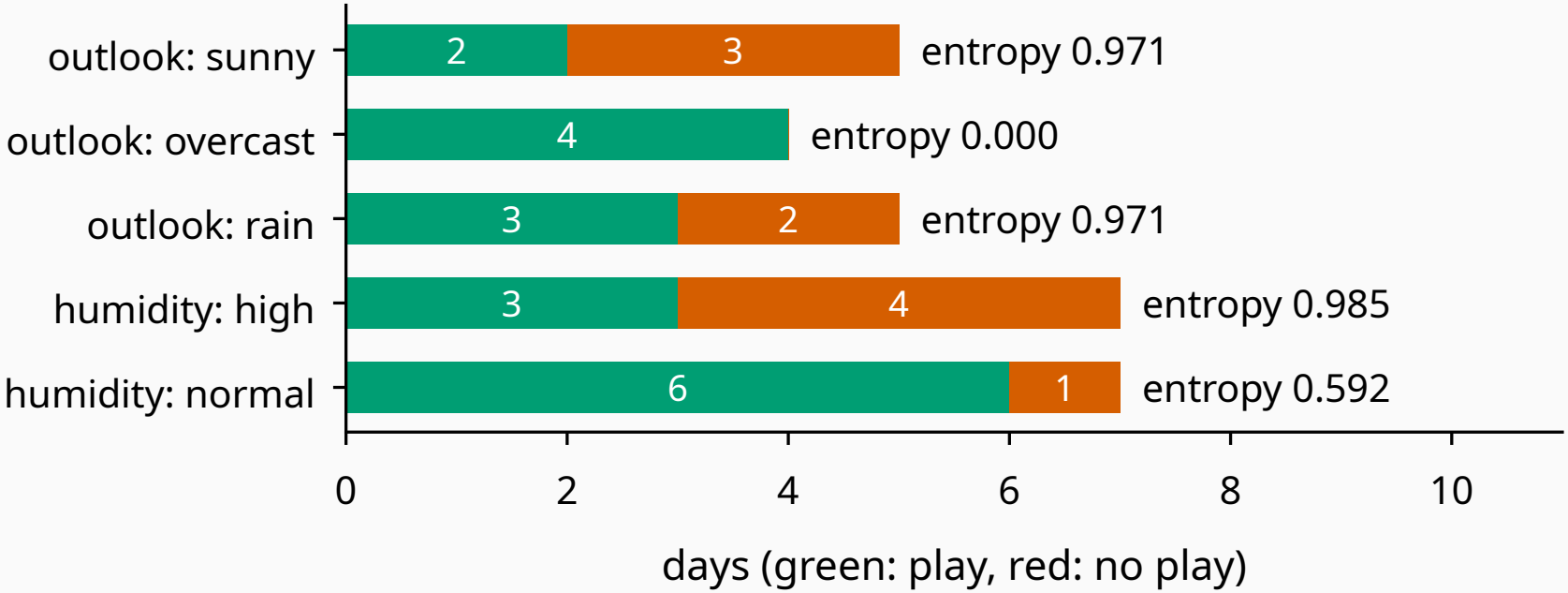


$$g = I_0 - \frac{n_L}{n} I_L - \frac{n_R}{n} I_R \quad \text{(definition of the gain)}$$

- Bigger groups weigh more; the gain is never negative
- Gini, question B: $g = 0.5 - \frac{50}{80} \cdot 0.32 - 0 = 0.30$
- Question A: both children have $I = 0.375$, $g = 0.125$

Worked Example: Play Tennis? (1/2)

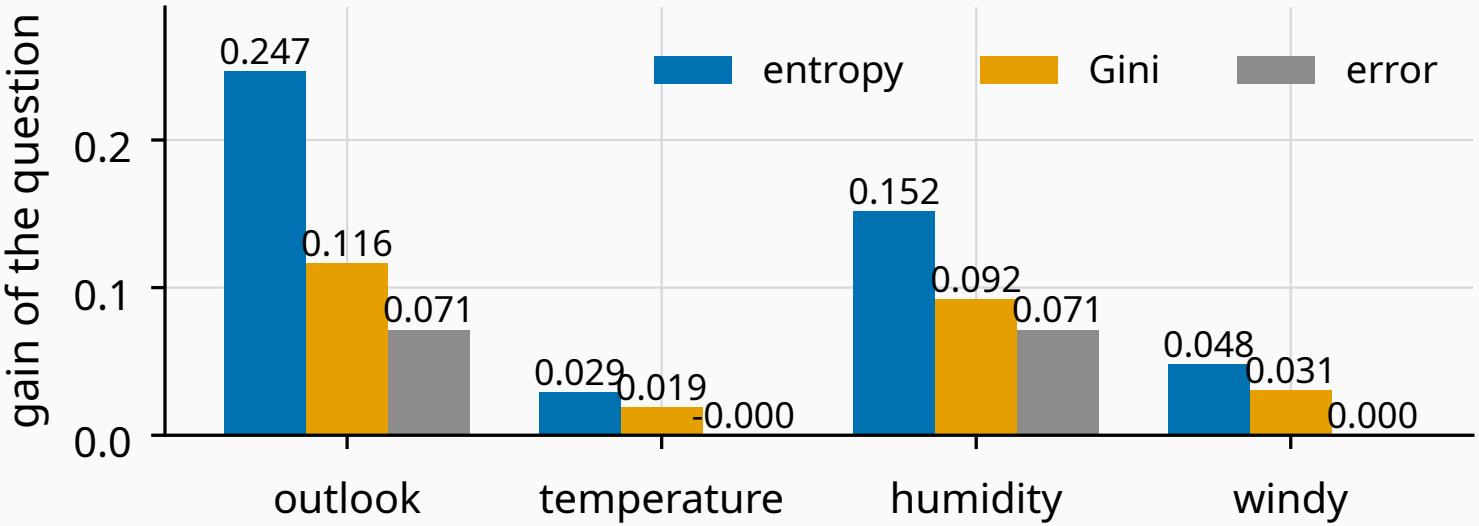
14 days: 9 play, 5 do not. Entropy of each group under two questions:



- Parent: $H = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$
- The *overcast* group is **pure**: always play

Worked Example: Play Tennis? (2/2)

The gain of **outlook** is the entropy before minus the weighted entropy after.

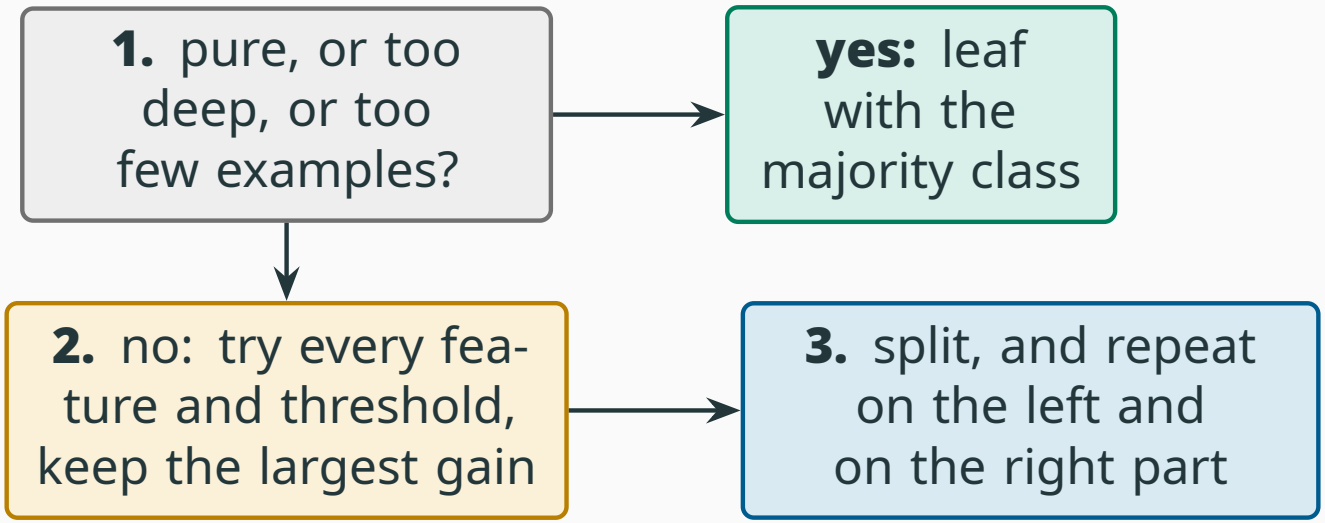


- Weighted entropy after:
  $\frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971 = 0.694$
- Gain: $0.940 - 0.694 = 0.247$
- Outlook is the root under all three criteria (demo T1)
- The *error* gives zero gain to two useful questions

Entropy or Gini?

- In practice they pick the **same** question almost always
- Entropy is a little more sensitive to small classes; Gini is cheaper
- Breast cancer: `worst_concave_points`, threshold 0.142 for both criteria
- scikit-learn uses Gini; C4.5 (Quinlan 1986) an entropy gain; Breiman's trees (Breiman et al. 1984) Gini
- Error is a poor *splitting* criterion but the right measure of the *final* tree

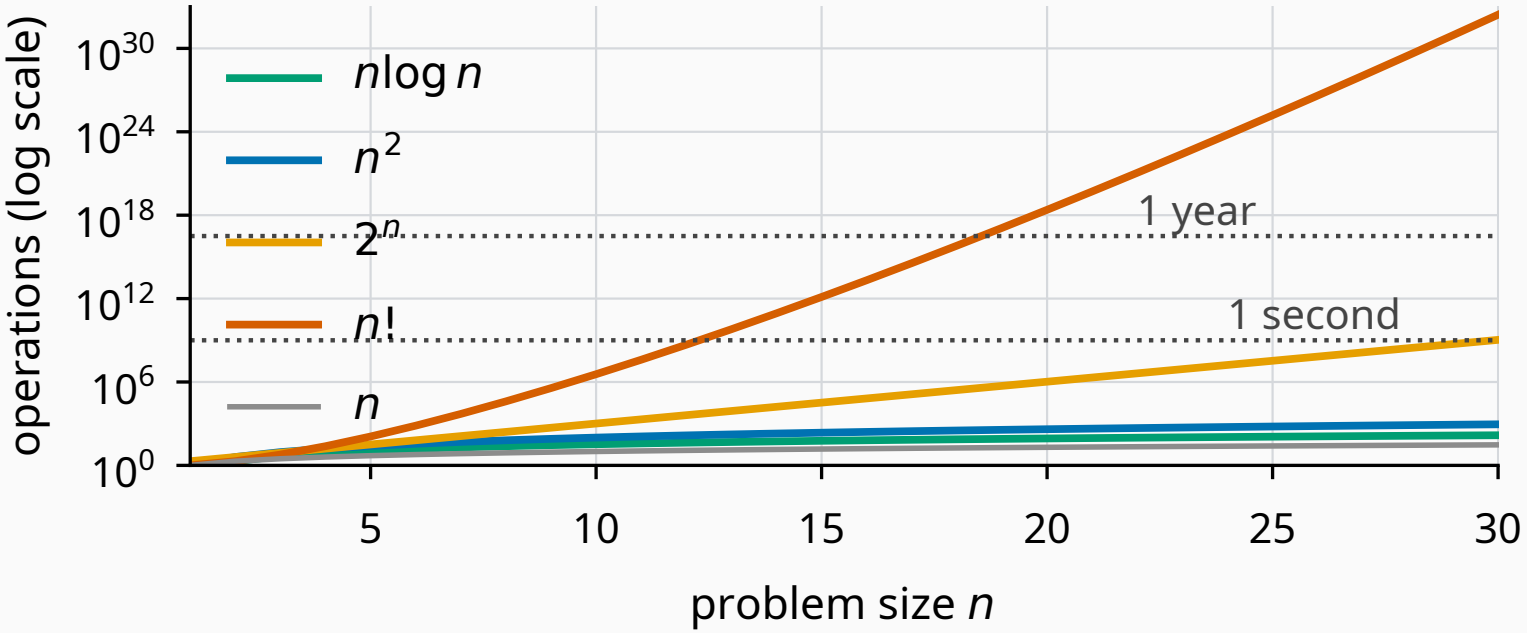
The Greedy Algorithm



- **Greedy:** each split is the best *now*, without looking ahead
- A very good question may come after a mediocre one (exclusive or)
- The same algorithm serves classes and numbers: only the impurity changes

How the Cost of an Algorithm Grows

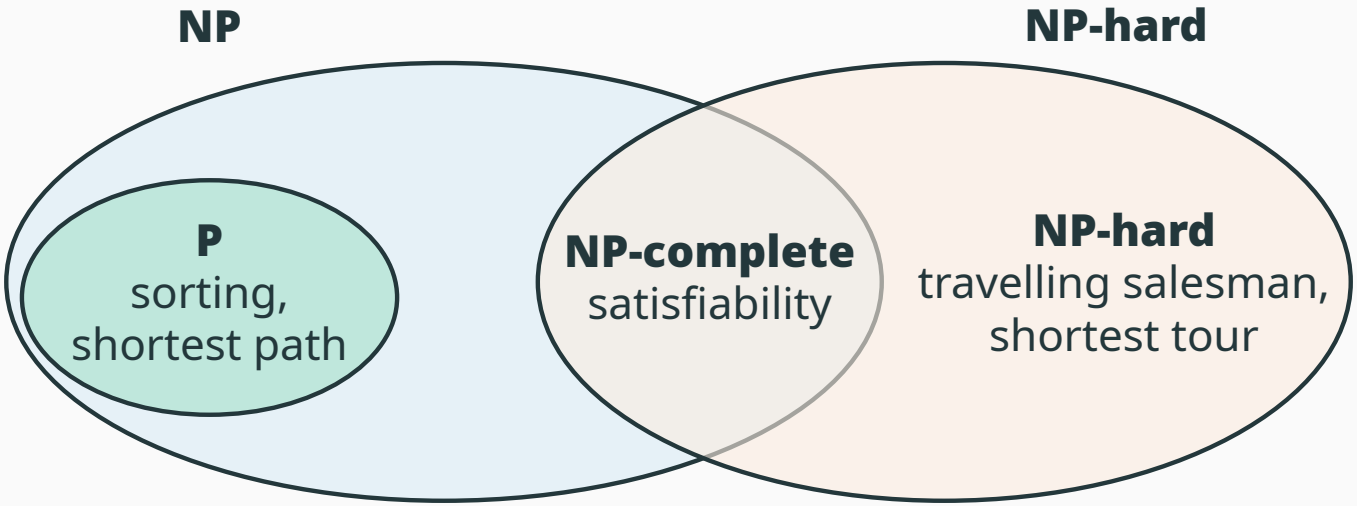
Why not build the best tree? Cost is the number of steps for an input of size n.



- n, $n \log n$, n^2 are **polynomial**; sorting is $n \log n$
- At 10^9 steps a second and $n = 50$: 2^n takes 13 days
- $n!$ takes 10^{48} years; gridlines are 10^6 apart

P, NP, NP-Complete, NP-Hard

Problems are sorted by how their cost grows and by how easily an answer is checked.



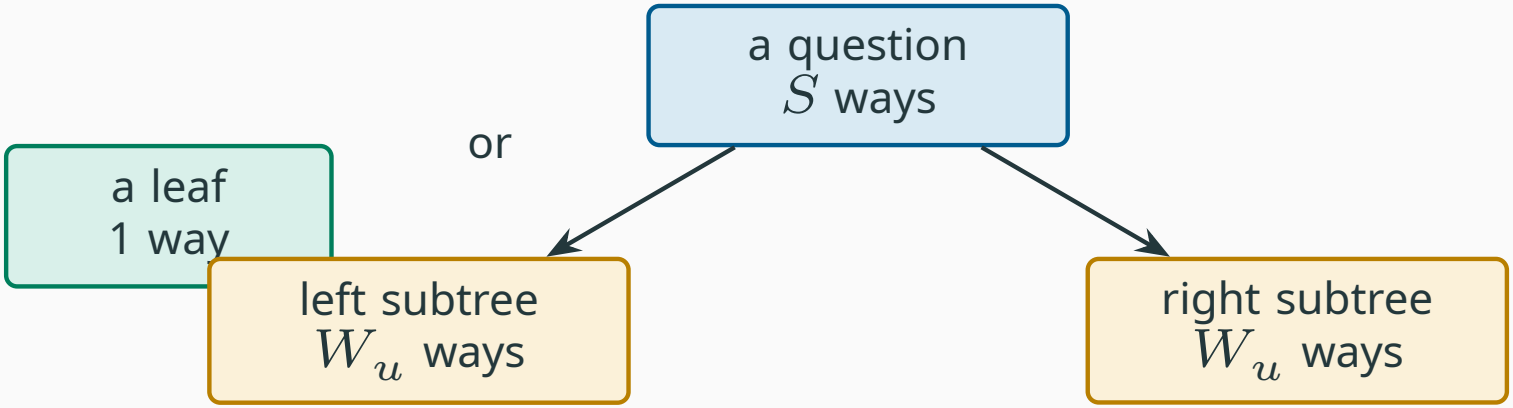
- **NP:** an answer is checked in polynomial time
- **NP-complete:** the hardest of NP: one fast algorithm solves all
- **NP-hard:** at least that hard, maybe outside NP
- **P versus NP is unsolved**

Notation: Counting

symbol	meaning	example
S	candidate questions (feature, threshold)	$S \approx d\,(n-1)$
u	levels of a tree	depth 2
W_u	trees with up to u levels	1 020 101 for $S = 100$, $u = 2$
W_{u+1}	trees with one more level	
v	levels of a category	10 purposes
U	a subset of the levels	{own, rent}
Q	questions "value in U?"	511 for $v = 10$

Counting the Trees: One Level More

A tree is a leaf, or a question with two subtrees, each one level shorter.

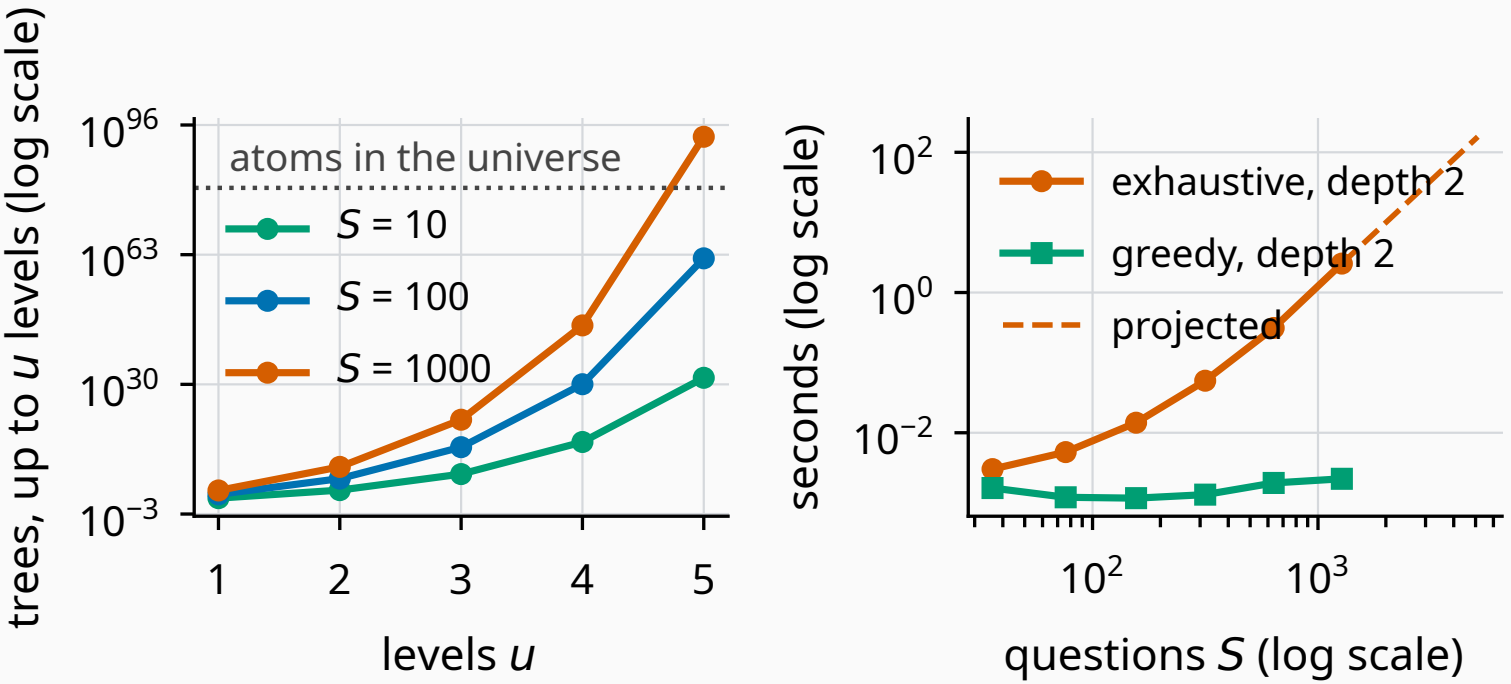


$$W_{u+1} = 1 + S\,W_u^2$$ (definition of the number of trees)

- One more level squares the count: a tower

The Number of Trees Explodes

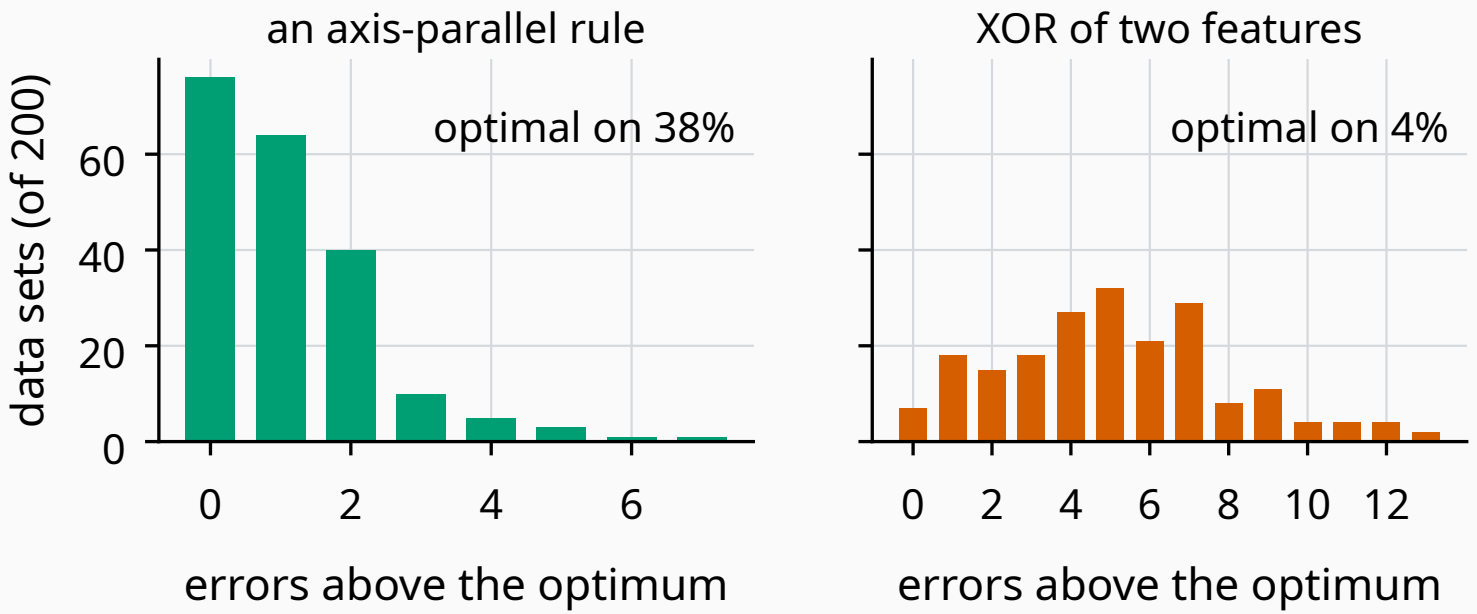
Trees with up to u levels for S questions, and the time of an exhaustive search:



- Depth 2, $n = 320$: 2 billion trees, 3 s (typical)
- Depth 3, $n = 40$: about 39 days

Greedy: A Good Tree at a Low Cost

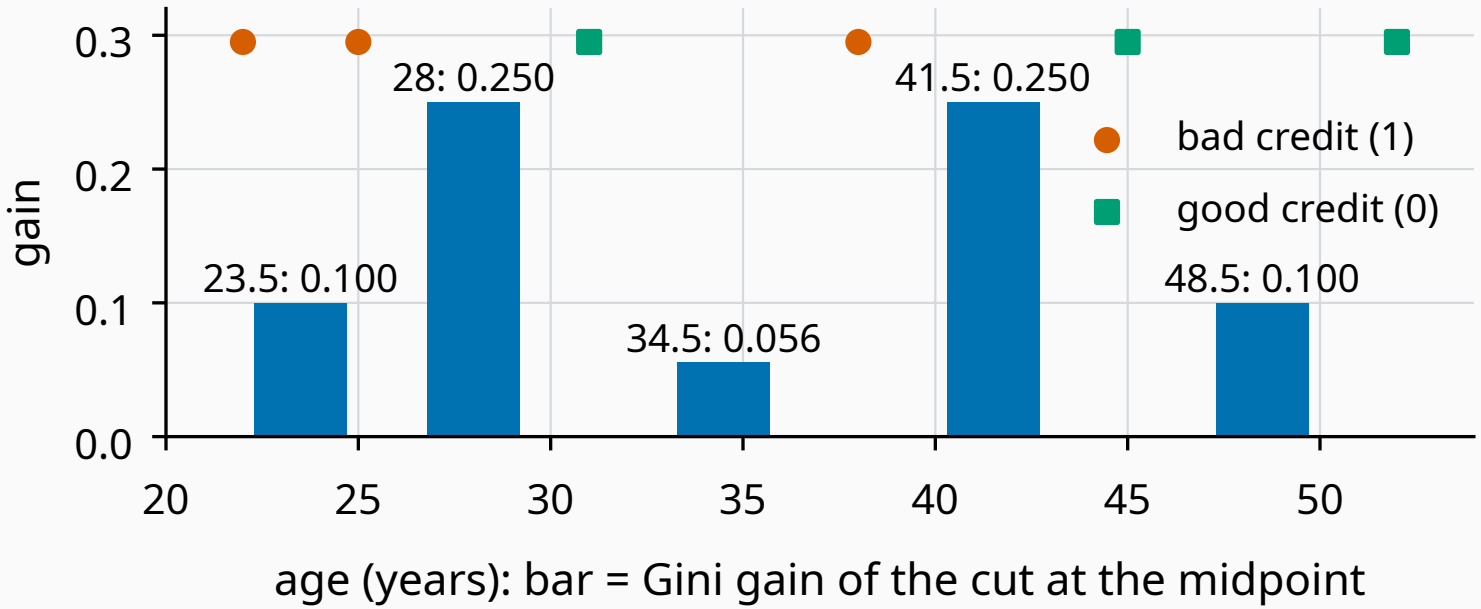
Finding the best tree is NP-complete (Hyafil and Rivest 1976); greedy is cheap and usually close.



- A rule, 200 sets: optimal on 76, within 2 errors on 180
- **Exclusive or fails:** 9.54 errors against 4.44; test error 0.467
- Greedy depth 2: 0.002 s (typical)

Numeric Features: Where to Cut?

The candidate questions are "$x \leq t$", with t midway between consecutive sorted values.



- Six applicants, bad credit $= 1$: gain of each cut
- Two cuts tie at 0.250 (28 and 41.5): either is a correct tree

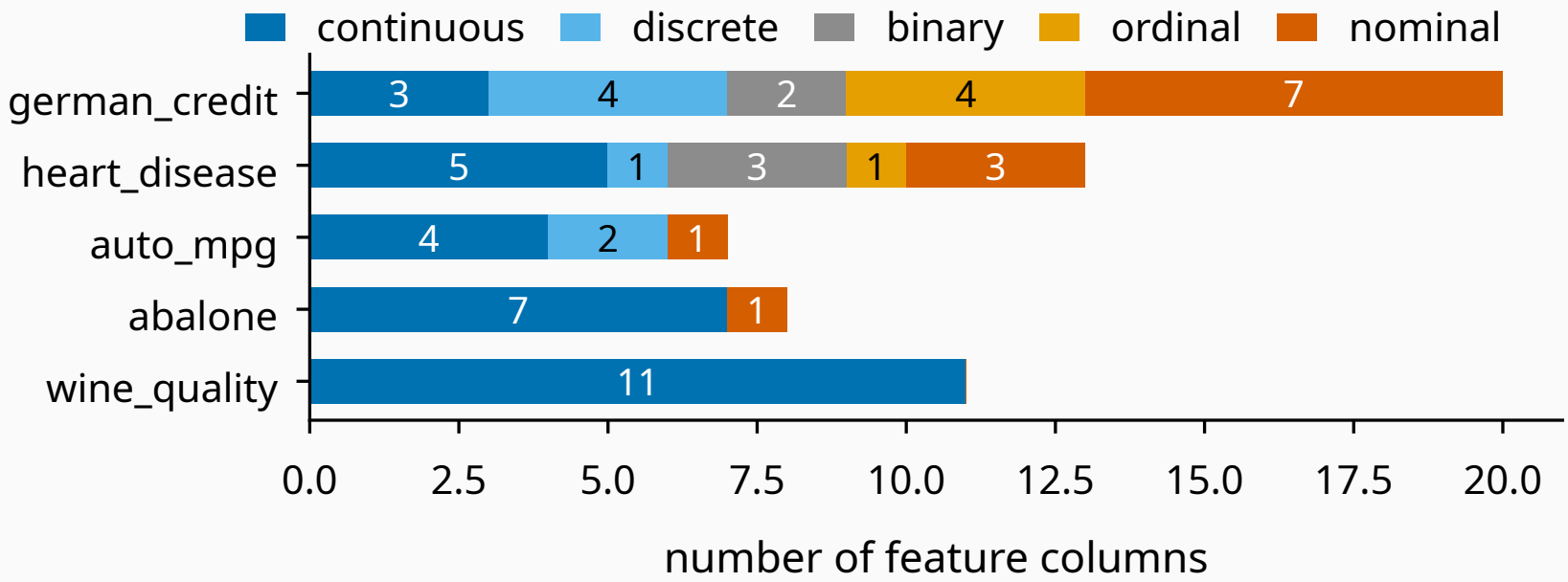
One Sorted Scan Finds the Best Cut

Sort once, then walk along the values with **cumulative class counts** on the left.

- The right counts are the totals minus the left counts
- Cost: one sort, $O(n \log n)$, then a linear pass per feature
- Without cumulative counts each cut needs a pass: $O(n^2)$
- A tie is broken by the implementation: two correct trees can differ
- Breast cancer, `worst_perimeter`: 513 candidate cuts, the best at 106.0

Why the Type of a Column Matters

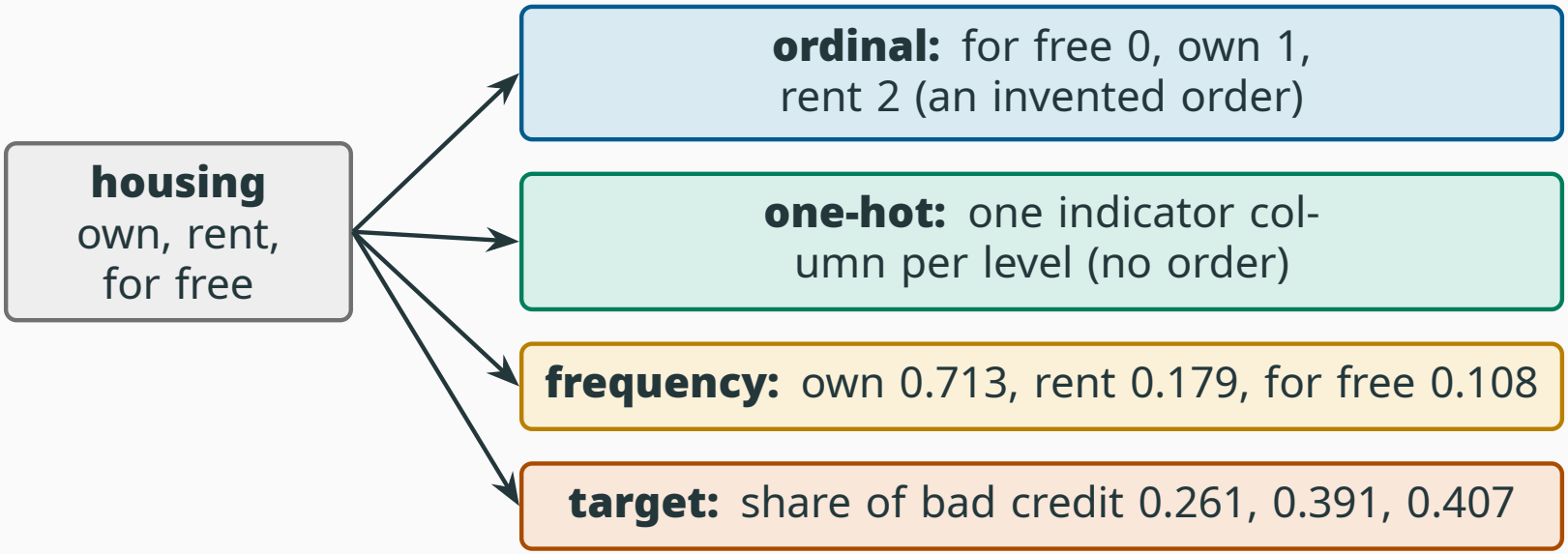
The type of a column decides which questions a tree may ask and which distance neighbours may use.



- **Numeric:** continuous (age) or discrete (dependents)
- **Ordinal:** ordered, gaps unknown (years employed)
- **Nominal:** no order (postcode); **binary**: two levels

Encoding a Category as Numbers

An **encoding** turns each level of a category into numbers. Example: the housing of one applicant.



- A tree asks "$\leq$ threshold": order matters, gaps do not
- **Target** encoding uses the class: fit it inside each training fold

A Tree Asks: Is the Value in a Subset?

A question on a category with v levels asks "is the value in U?". Example: $v = 3$.

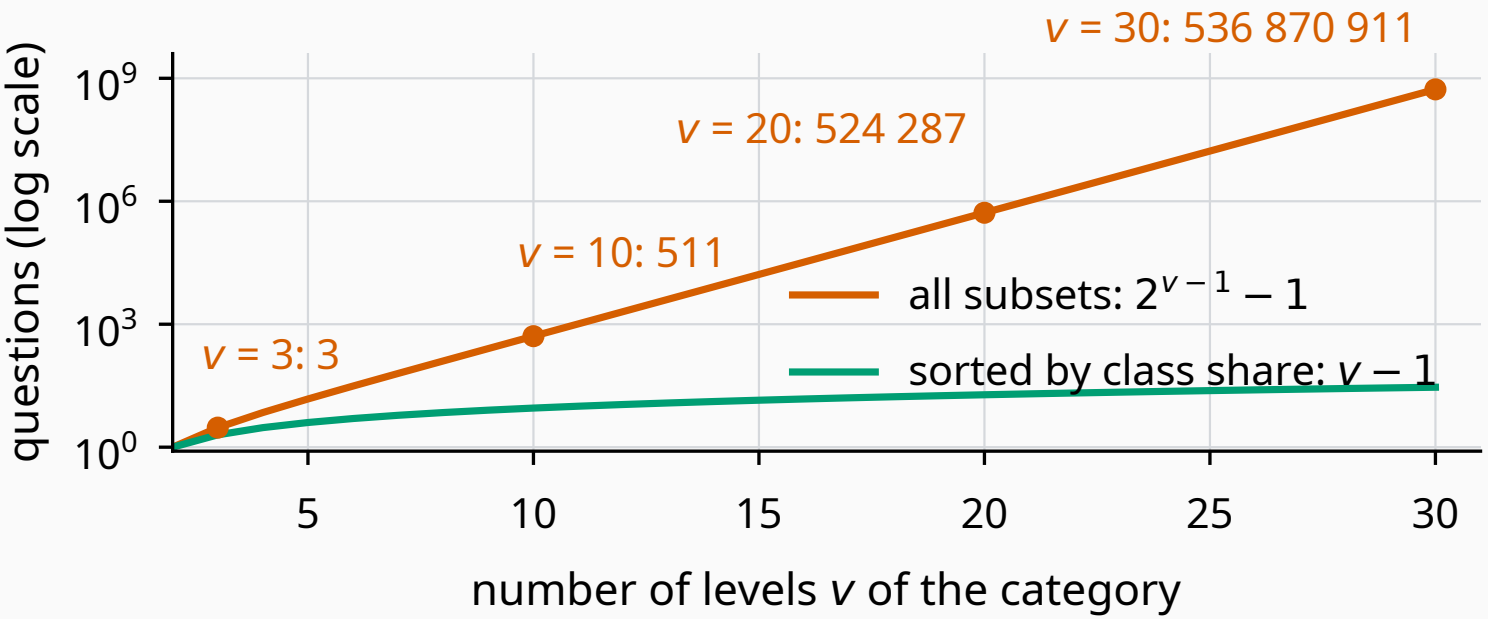
U	**not** U
own	rent, for free
rent	own, for free
for free	own, rent

$$
\begin{aligned}
Q &= \frac{2^v - 2}{2} && \text{(definition of the number of questions)} \\
&= 2^{v-1} - 1 && \text{(simplify)}
\end{aligned}
$$

- 2^v subsets, less empty and full, halved

Many Levels, Many Questions

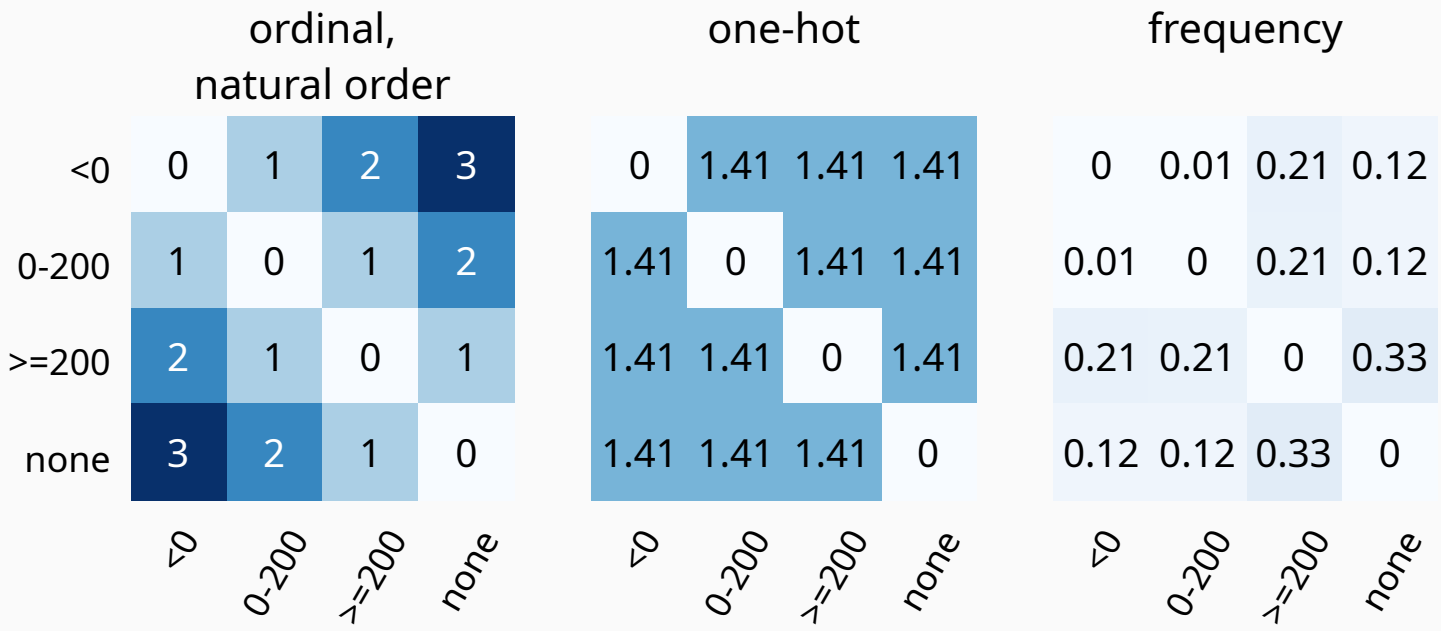
The number of questions doubles with each level.



- Sorted by class share: $v - 1$ cuts, same best gain (Breiman et al. 1984)
- Purpose, $v = 10$: 511 or 9 questions, gain 0.012

KNN Needs a Distance Between Categories

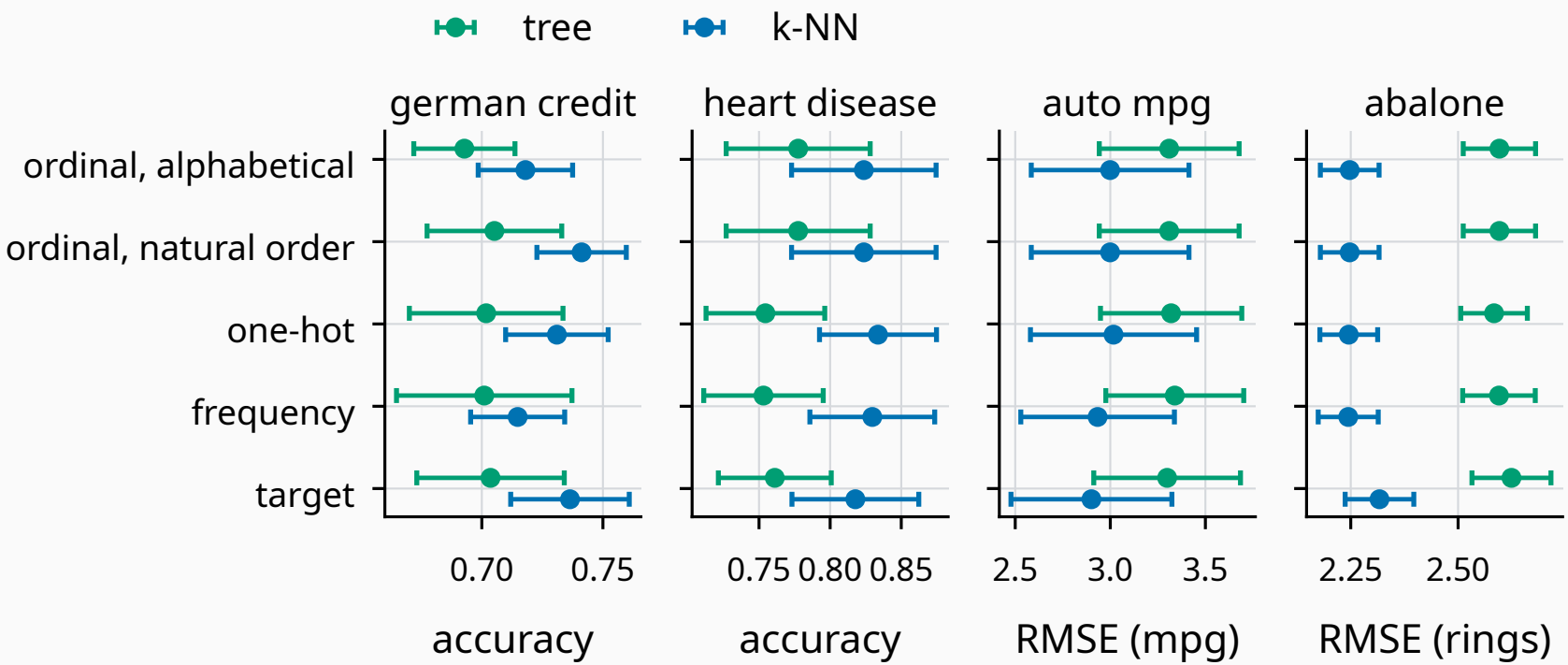
Neighbours add up differences, so a category needs a distance. German credit, checking account:



- Ordinal codes invent distances: "< 0" is 3 from "none"
- One-hot: every pair is $\sqrt{2}$ apart (**Hamming**)
- **Gower:** mean of per-column differences; two applicants: 0.468

Does the Encoding Change the Accuracy?

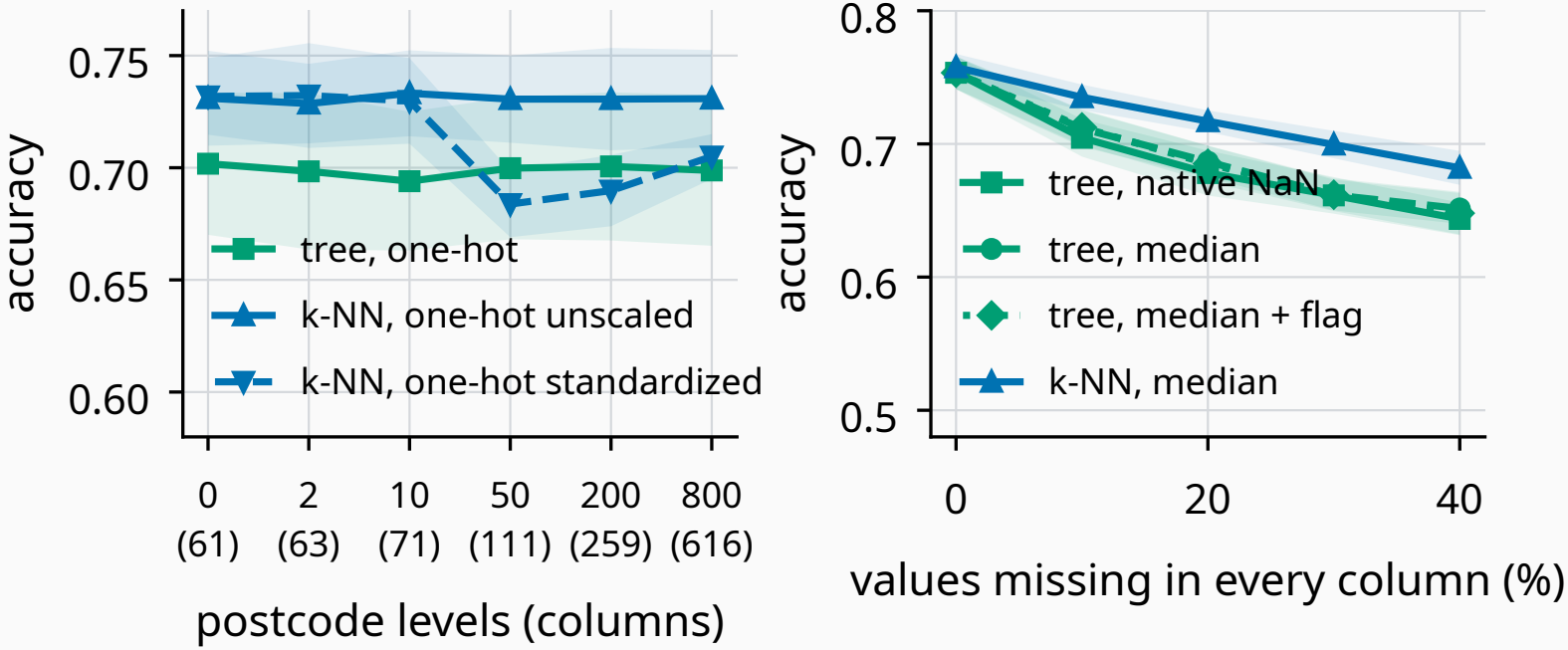
Five encodings, a tree and neighbours, four data sets: dots are means, bars the spread.



- German credit tree: 0.693 to 0.705, spread 0.02 to 0.04
- Neighbours: natural order 0.741, alphabetical 0.718

Many Levels and Missing Values

Left: a random postcode with v levels added to German credit. Right: values removed from wine; impute or flag.



- Scaled one-hot: neighbours 0.684 at $v = 50$ (unscaled 0.731)
- Wine, 40% missing: tree 0.651, neighbours 0.682

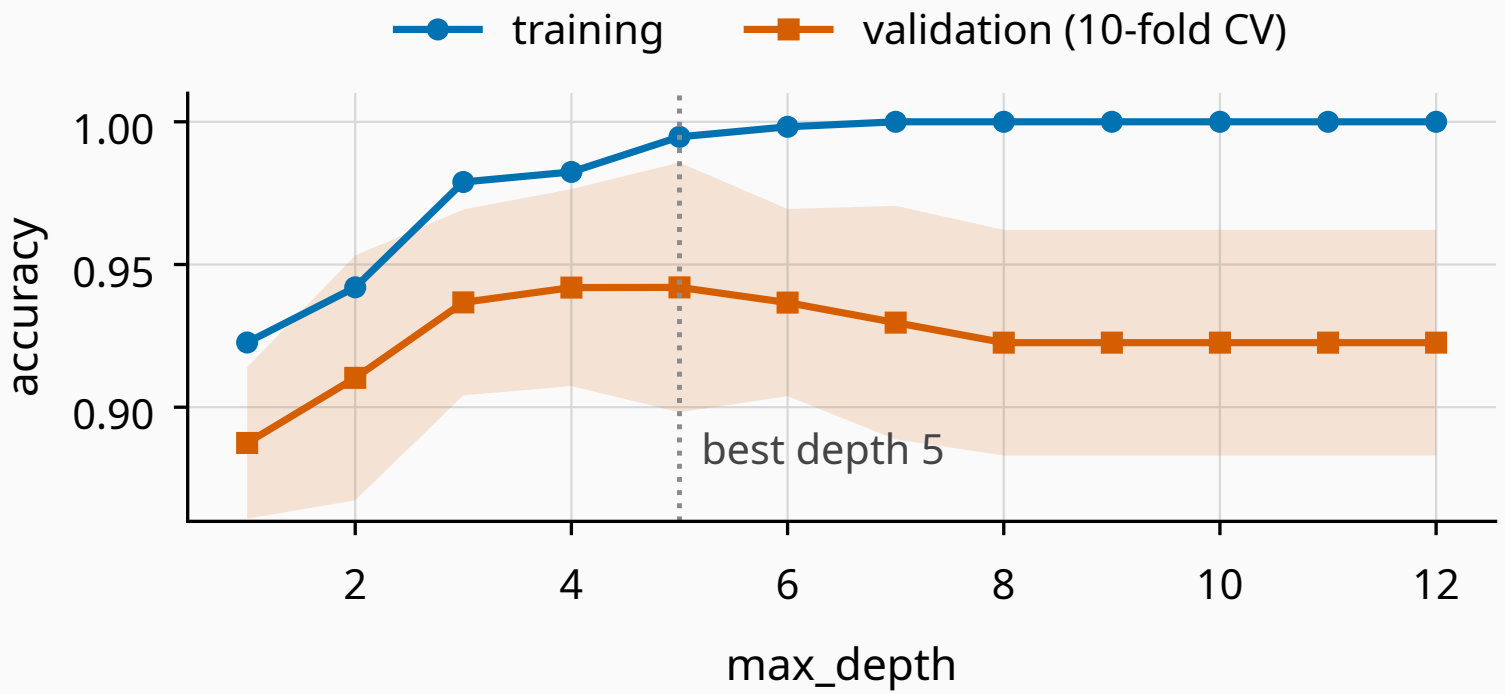
When to Stop

A tree grown until every leaf is pure has **100% training accuracy** and fits the noise. Stopping rules:

- The node is **pure**: nothing to gain
- **Maximum depth** reached (`max_depth`)
- **Examples per leaf** (`min_samples_leaf`): enough to trust a leaf
- **Minimum gain** (`min_impurity_decrease`): the question is not worth asking
- These are *hyperparameters*: choose them on **validation data** (Class 01)

Depth and Overfitting

Breast cancer, accuracy against the maximum depth: training data, and 10-fold cross-validation.



- Training accuracy reaches 1.000 at depth 7
- Validation peaks at **depth 4-5 (0.942)**, then falls to 0.923

Depth Depends on the Data

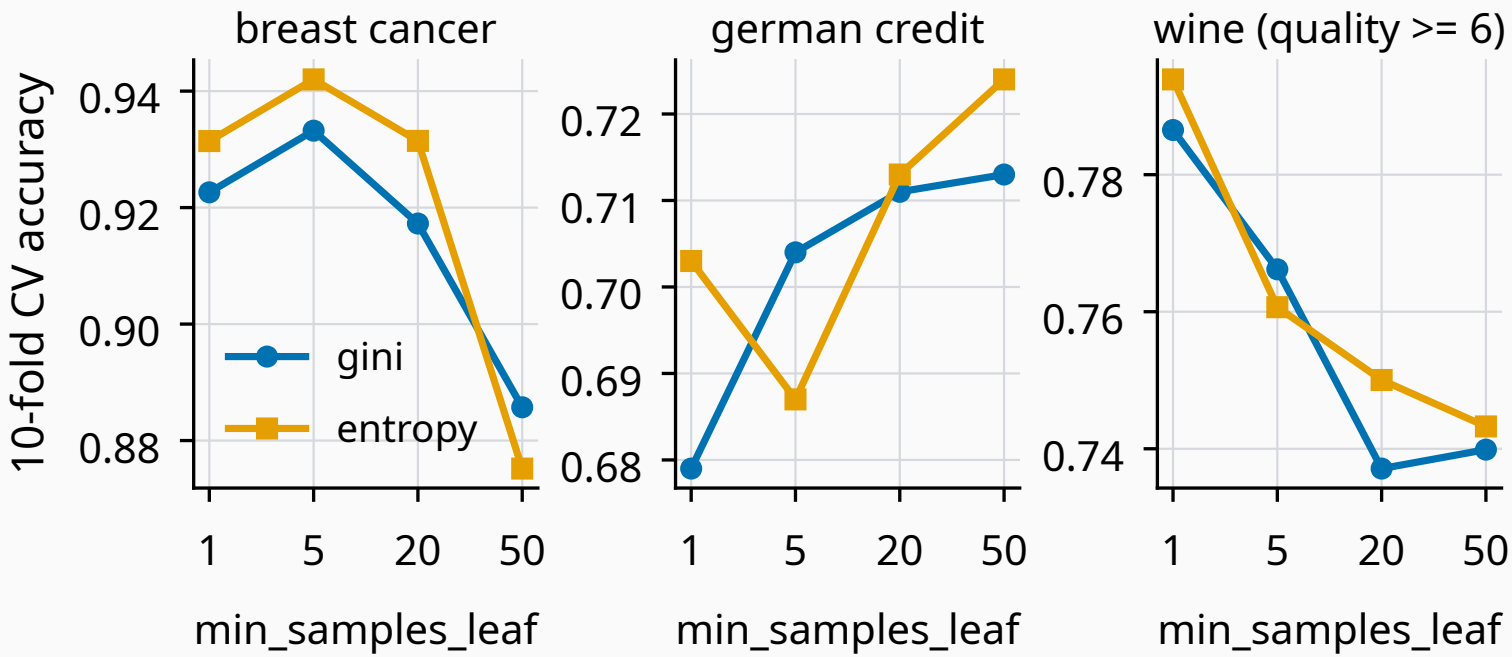
Best depth by 10-fold cross-validation, and an unlimited tree (depth 15):

data set	best depth	CV accuracy at best	train, depth 15	CV, depth 15
breast cancer	5	0.942	1.000	0.923
German credit	3	0.717	1.000	0.685
wine (≥ 6)	15	0.780	0.978	0.780

- Small, messy data want a **shallow** tree; large, clean data use depth
- The gap training — validation is the symptom: 0.315 for German credit

Pre-Pruning: the Leaf Size

`min_samples_leaf`: a leaf with fewer examples is not allowed. Cross-validated accuracy:



- The best value depends on the data
- The criterion matters little

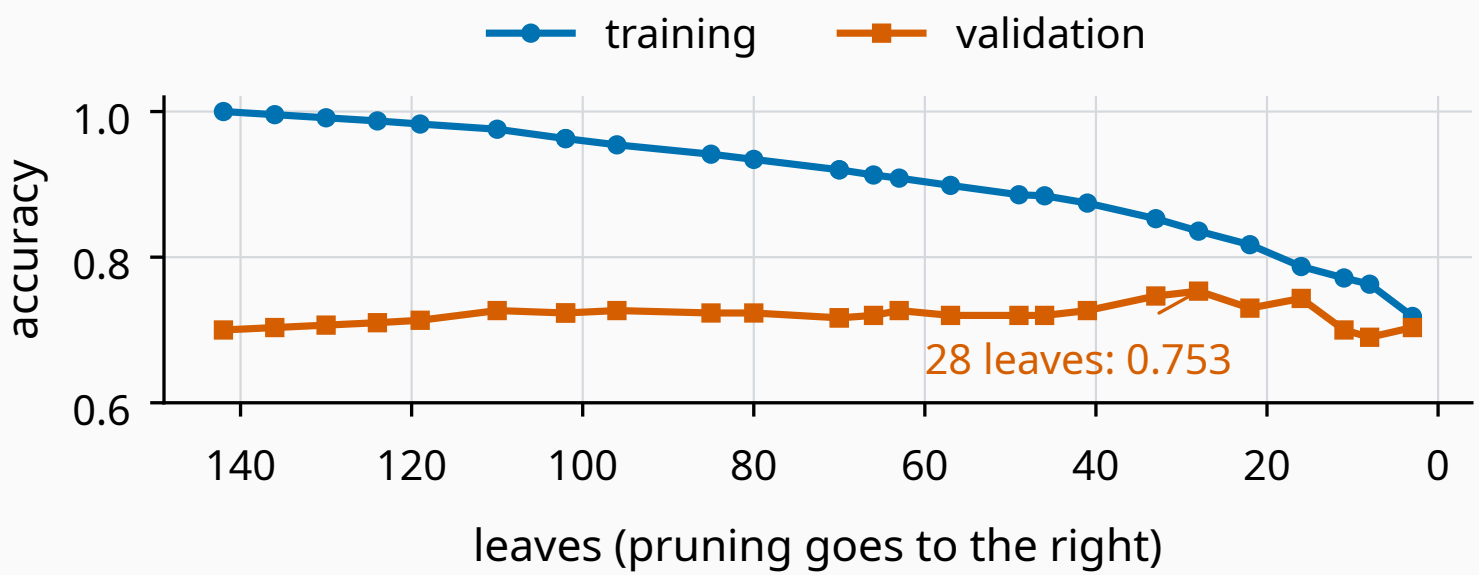
Notation: Pruning and Regression

symbol	meaning	used in
λ	price of one leaf	pruning
$\lvert T \rvert$	number of leaves of the tree T	pruning
E_T	training error of the tree T	pruning
C_λ	cost of T: error plus the price of its leaves	pruning
R	a leaf of a regression tree (a cell)	regression
y_i	targets: target of example i, not its prediction	regression
i	index of an example	regression
$\bar{y}_R$	mean of the targets in R	regression
E_R	squared errors of the targets around $\bar{y}_R$	regression

Post-Pruning: Cost-Complexity

Grow the full tree, then cut back with a price per leaf, as ridge prices weights.

$$C_\lambda = E_T + \lambda \left|T\right| \quad \text{(definition of the penalized error)}$$

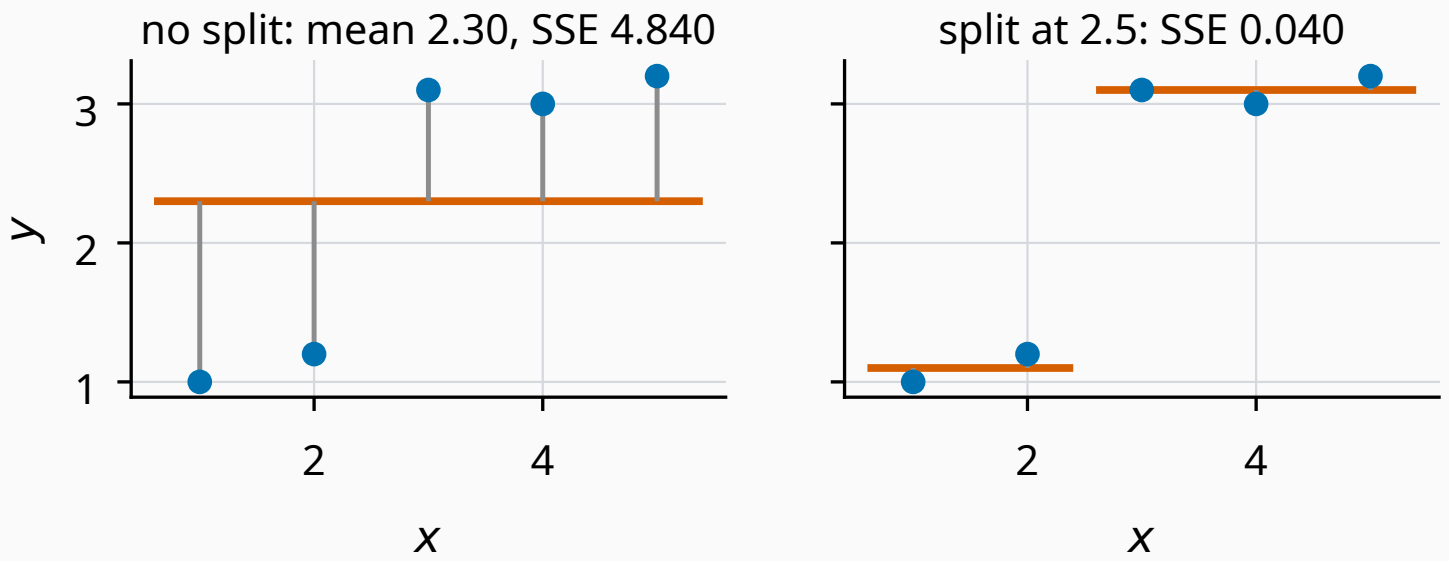


- A larger λ prunes the cheapest branch first

Regression Trees

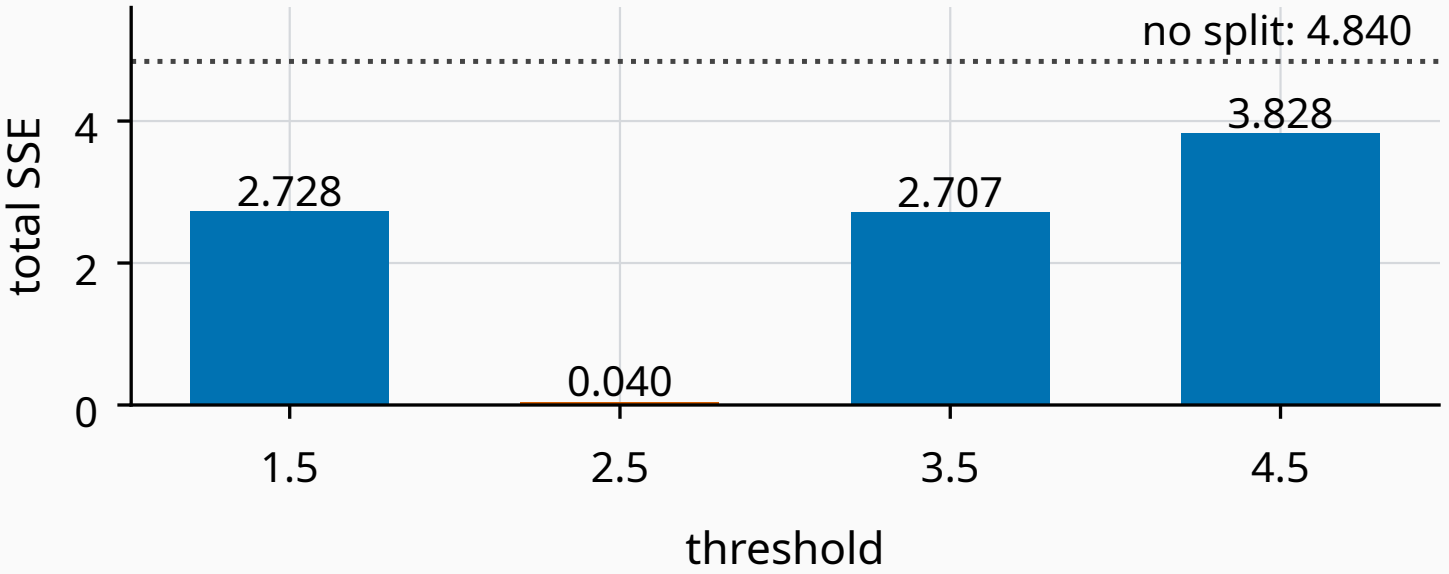
A leaf predicts the **mean**; questions are judged by squared error removed.

$$E_R = \sum_{i \in R} (y_i - \bar{y}_R)^2 \quad \text{(definition of the squared error of a leaf)}$$



A Regression Split by Hand

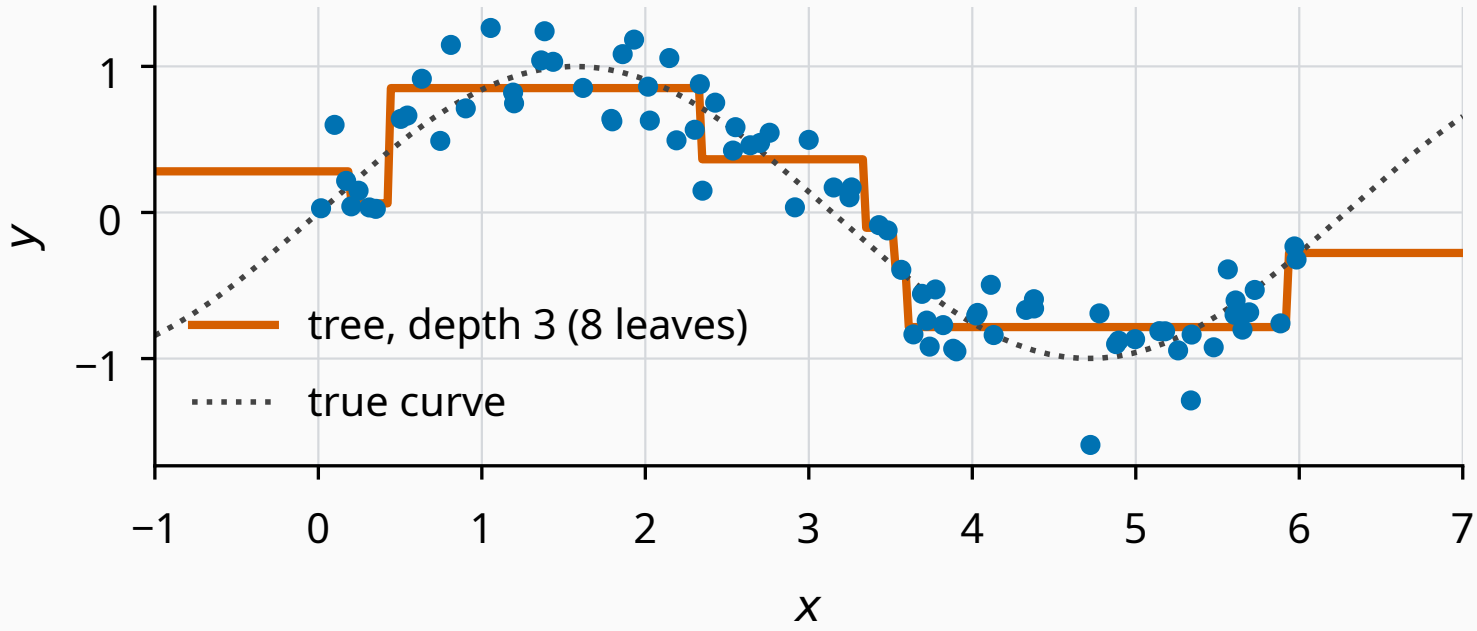
Five points: without a split, the mean is 2.30 and the squared error 4.84.



- The cut at 2.5 gives means 1.10 and 3.10
- Gain: parent minus children, $4.84 - 0.040 = 4.80$

The Prediction Is a Staircase

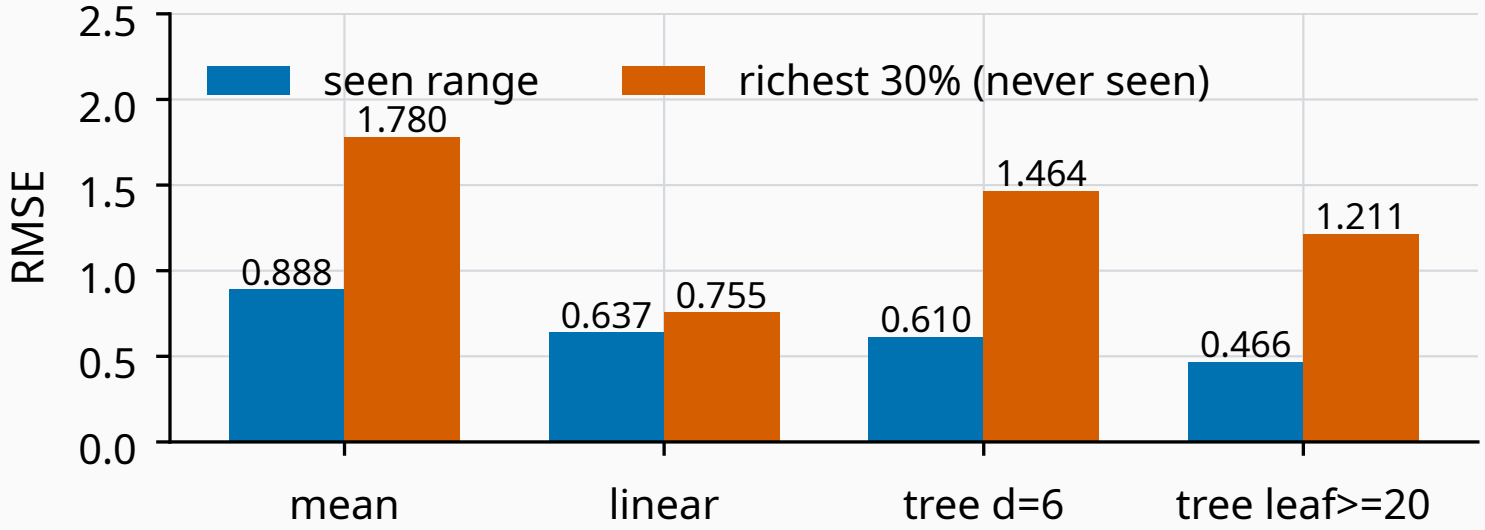
A tree of depth 3 on a noisy sine curve:



- Depth 3: **8 leaves**, 8 different values
- Outside the data the tree repeats its first or last value

A Tree Cannot Extrapolate

Train California housing on the 70% of districts with the lowest income; test on the **richest 30%**.



- Inside the data the tree beats the line: 0.466 against 0.637
- Outside it is **much worse**: 1.211 against 0.755
- A tree predicts a constant; the line continues the trend

Regression Trees on Real Data

5-fold root mean squared error (concrete in MPa, abalone in rings, California in 100 000 dollars):

data set	mean	linear	tree $d = 3$	tree $d = 6$	tree $d = 12$	leaf ≥ 10
concrete	16.70	10.43	10.55	7.71	**6.75**	7.21
abalone	3.22	**2.21**	2.50	2.35	2.80	2.39
auto-mpg	7.79	3.39	3.58	3.65	3.83	**3.18**
California	1.15	0.67	0.80	0.71	0.73	**0.65**

- Concrete has strong **interactions**: the tree wins by a wide margin
- Depth 12 is best on concrete and worst on abalone: **validate**

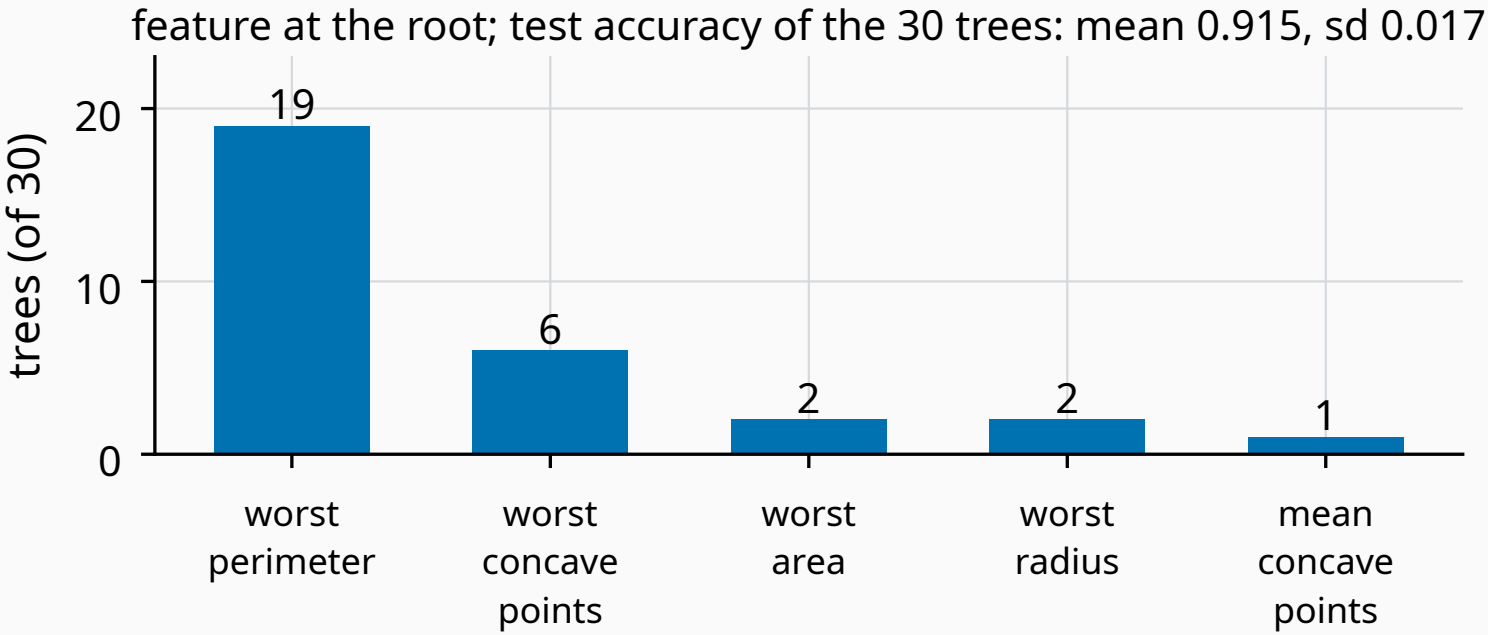
Summary: Strengths and Weaknesses of Trees

strengths	weaknesses
readable rules, easy to explain	high **variance**: other data, other tree
no scaling, no order to encode	axis-parallel borders: diagonals are staircases
numeric and categorical features	cannot extrapolate (regression)
captures interactions and non-linearity	greedy: not the best tree
fast to train and to predict	biased importances

- The weakness that matters most is the **variance**: next class, ensembles

Instability

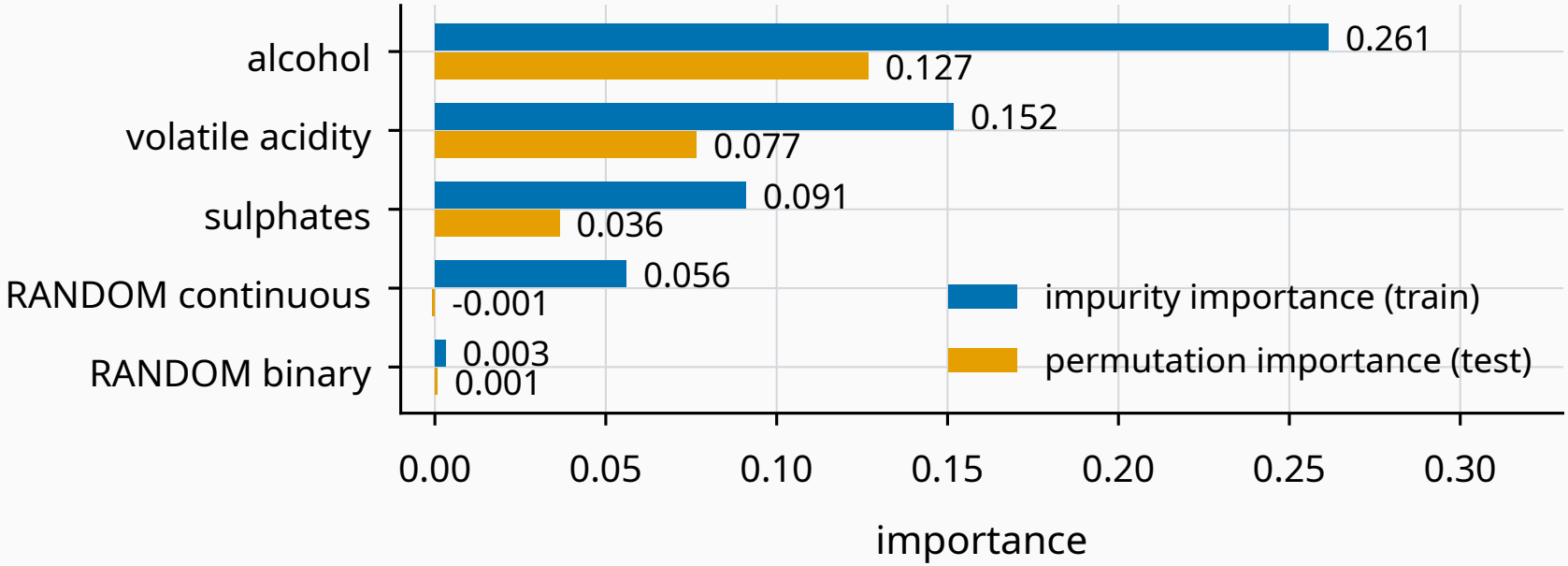
Breast cancer: 30 trees, each grown on a **bootstrap sample** (n cases drawn *with* replacement).



- Test accuracy of the 30 trees: 0.871 to 0.947; 6 to 18 leaves
- The *majority vote*: 0.918, a first look at **bagging**
- Do not read too much into *the* first split

Feature Importances and Their Bias

Impurity importance: the total gain of a feature's splits.
Wine plus a random column:



- The random column gets 5.6% of the importance
- **Permutation importance** sees through it
- Copies **split** the credit: 0.261 becomes $0.027 + 0.234$

DEMO T1–T7: Notebook

`01_decision_trees.ipynb` (B1, T1 to T7) **shows** regions, gain, pruning, encodings, counting

Look for: where each model fails, and how results move

k-Nearest Neighbours

Learning by Analogy

How does a doctor, or a real-estate agent, estimate? “Cases like this one were …”. Nearest neighbours do exactly that.

- **No training:** the “model” is the stored data; learning is *lazy*
- To predict, find the k **most similar** cases: they **vote** (class) or **average** (number)
- The **distance** (what is similar) and k (how many) define the learner
- A very old idea (Cover and Hart 1967); still a strong baseline
- The *analogizer* of Project 1, with the support vector machine of Class 05

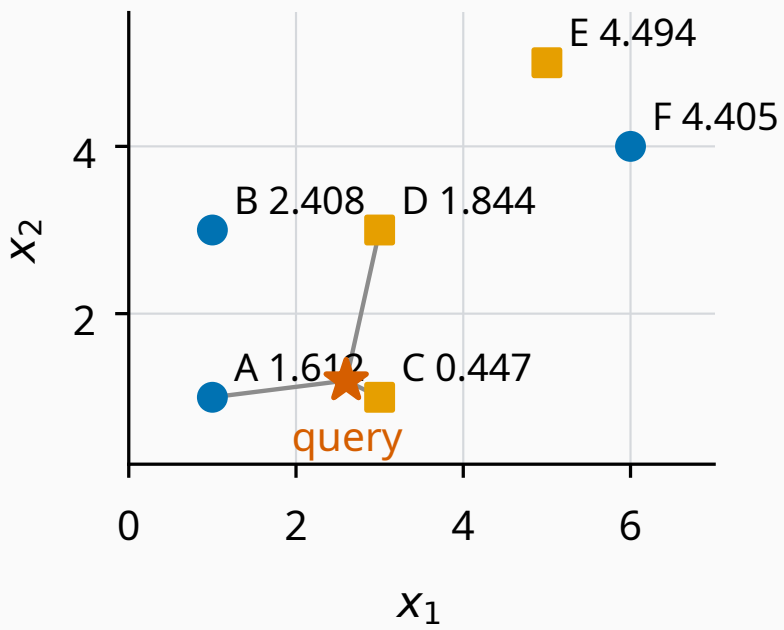
The Algorithm in Four Steps

1. **Standardize** the features (training statistics only; Class 03)
2. **Distance:** from the new case to every training case
3. **Neighbours:** keep the k smallest distances
4. **Combine:** majority vote, or the mean of the targets

- Nothing is fitted: all the cost is paid **at prediction time**, $O(n\,d)$ per case
- The decision boundary is never written down: the data imply it

Worked Example: Six Cases, One Query

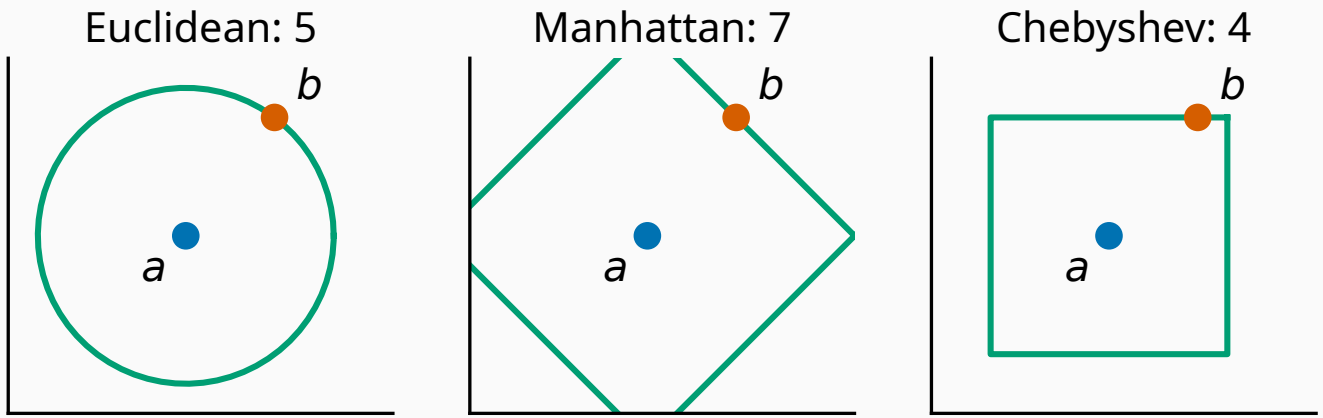
Query $q = (2.6, 1.2)$, six cases A to F with classes 0 and 1:



- $k = 1$: C $\rightarrow$ class **1**
- $k = 3$: C, A, D $\rightarrow$ votes 2 to 1: class **1**
- $k = 5$: C, A, D, B, F $\rightarrow$ votes 2 to 3: class **0**
- The answer **changed** with k

Distances

For two cases $a = (1, 2)$ and $b = (4, 6)$, the shape of "equally far" depends on the distance:



- Euclidean 5, Manhattan 7, Chebyshev 4, Hamming 2 (differing coordinates)
- Cosine: $1 - \frac{16}{2.24 \cdot 7.21} = 0.008$, the angle, not the length
- Euclidean is the default; cosine suits text; Hamming suits categories

Which Distance? (Demo K1)

Test accuracy of $k = 5$ neighbours, standardized features:

data set	Euclidean	Manhattan	Chebyshev	cosine
breast cancer	0.951	**0.958**	0.944	0.930
digits	**0.973**	0.958	0.938	0.960
spam	0.932	**0.938**	0.912	0.933
heart disease	0.803	0.803	0.750	**0.829**

- No distance wins everywhere; 1 to 3 points are chance
- The distance is a **modelling decision**: it says what "similar" means

Why Scaling Is Essential

Two patients: a = (age 30, income 30 000), b = (age 40, income 30 500).

- Raw squared distance: $10^2 + 500^2$: the **income decides**, age counts 0.04%
- Standardized (sd 10 years, 10 000 euros): $1^2 + 0.05^2 = 1.0025$: **age decides**
- Neither is "right": scaling gives each feature the **same chance**
- Rule of Class 03: statistics from the **training** part only

Scaling in Numbers (Demo K2)

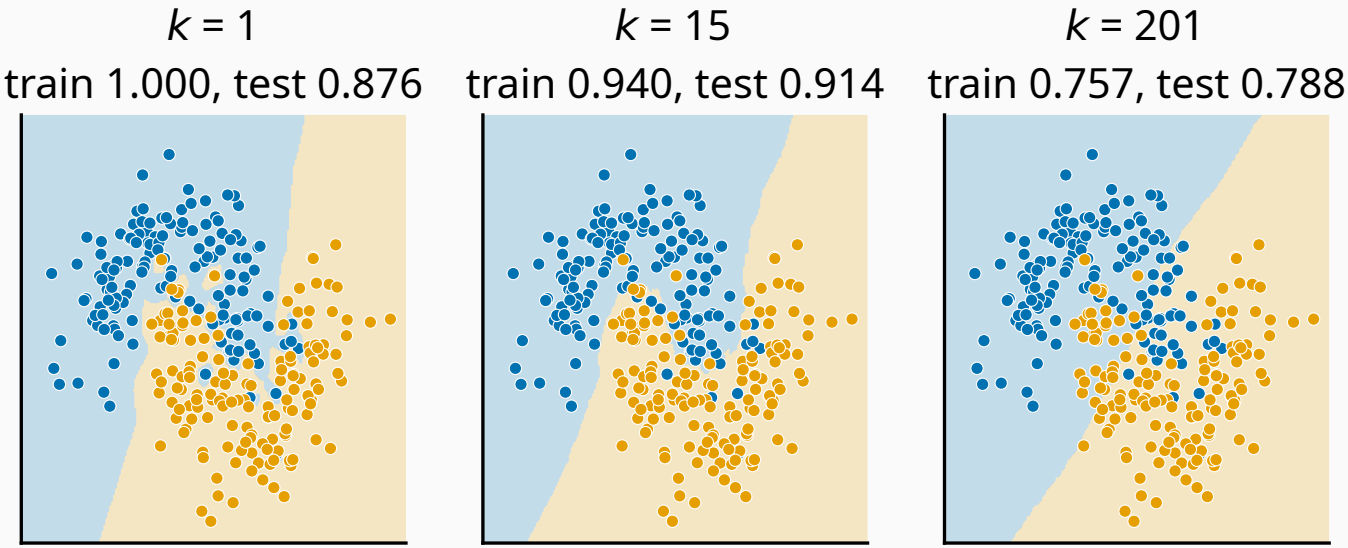
Test accuracy, $k = 5$, raw against standardized features:

data set	raw	standardized	range ratio (largest / smallest)
breast cancer	0.916	**0.951**	140 569
wine (≥ 6)	0.676	**0.763**	8 367
German credit	0.616	**0.692**	18 174
heart disease	0.618	**0.803**	438
digits	**0.980**	0.973	16
spam (log1p)	0.929	0.932	8

- The wider the ranges, the more standardization matters: up to **18 points**
- Digits: 64 pixels on the **same** scale; standardizing slightly hurts

Choosing k: the Bias-Variance Knob

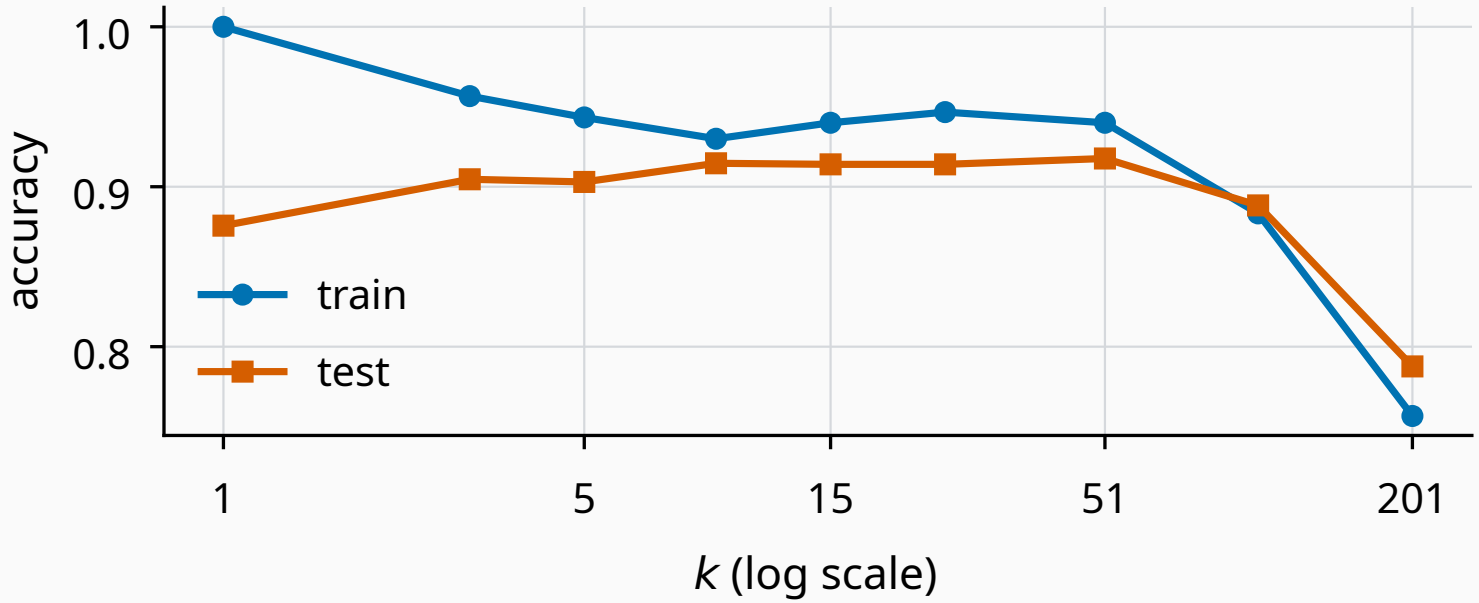
Two moons, 300 training cases: the border for $k = 1, 15, 201$.



- $k = 1$**:** follows every case, noise included: zero training error
- **Large** k**:** a smooth border, then the majority class
- Rule of thumb: $k \approx \sqrt{n}$, odd for two classes, chosen on validation data

The Curve of k

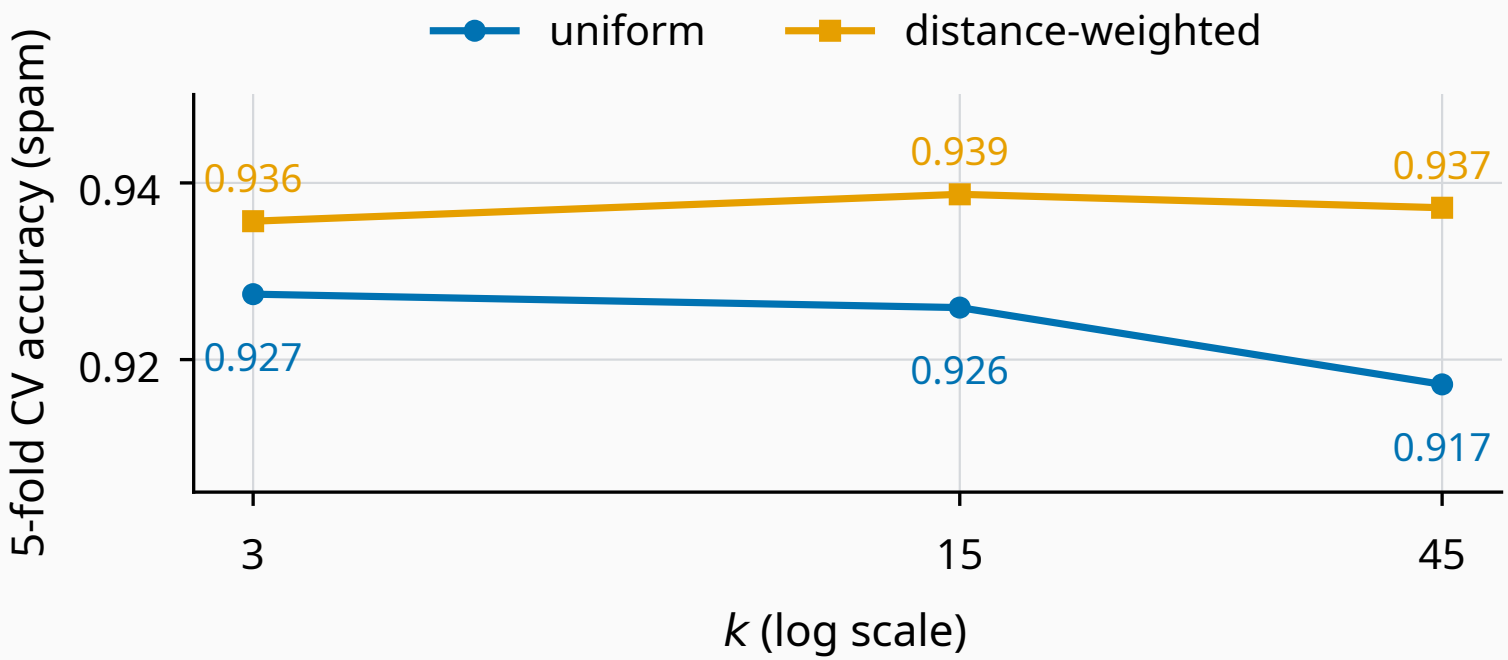
Accuracy on the two moons against k, training and test:



- Best k by cross-validation: **11** (breast cancer 0.967), **5** (digits 0.977), **9** (German 0.735)
- German credit: $k = 1$ gives 0.677, $k = 59$ gives 0.710

Weighted Voting

Let each neighbour vote with weight $w_i = 1/d_i$: near ones count more.



- Six cases, $k = 5$: weights 2.24 (C), 0.62 (A), 0.54 (D), 0.42 (B), 0.23 (F)
- Class 1: $2.24 + 0.54 = 2.78$; class 0: 1.26; so class **1**
- Ties become rare; the result depends less on k

KNN for Regression

The prediction is the **mean** of the k nearest targets (or the distance-weighted mean): a *local average*.

- A noisy sine, 80 points; error to the *true* curve:
- $k = 1$: 0.254; $k = 3$: 0.158; $k = 10$**: 0.109**; $k = 40$: 0.278
- $k = 1$ passes through every point; $k = 40$ flattens the peaks
- Like a tree: **no extrapolation**
- Unlike a tree: smooth, with no axis-parallel steps

KNN Regression in Numbers (Demo K4)

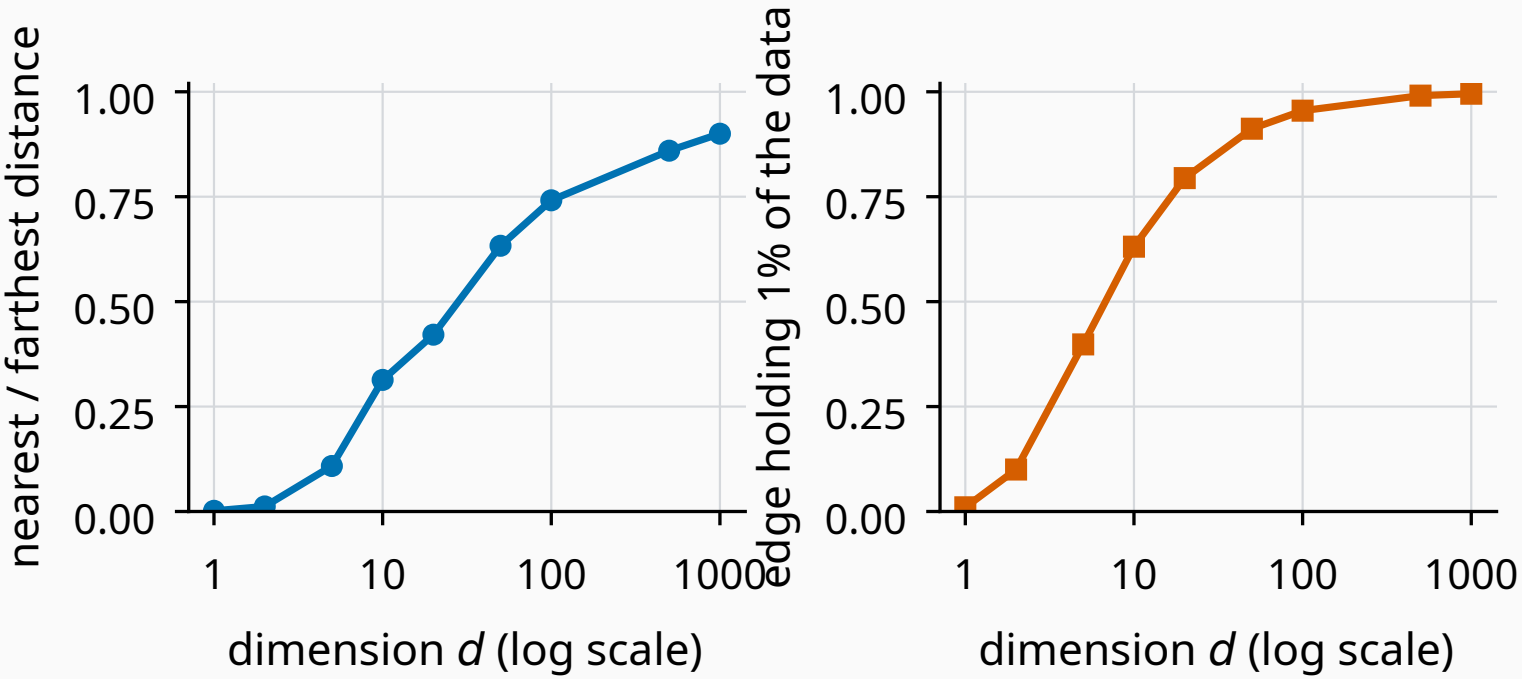
5-fold root mean squared error, standardized features:

data set	mean	linear	$k = 1$	$k = 5$	$k = 15$	$k = 15$ weighted
concrete	16.70	10.43	9.61	8.71	9.79	**8.48**
abalone	3.22	**2.21**	2.91	2.29	2.22	2.21
auto-mpg	7.79	3.39	3.41	3.09	3.16	**3.03**
California	1.15	0.67	0.81	0.64	0.62	**0.62**

- On concrete and California a local model beats the line
- On abalone (nearly linear, noisy) the line is as good; $k = 1$ is never best

The Curse of Dimensionality (Demo K3)

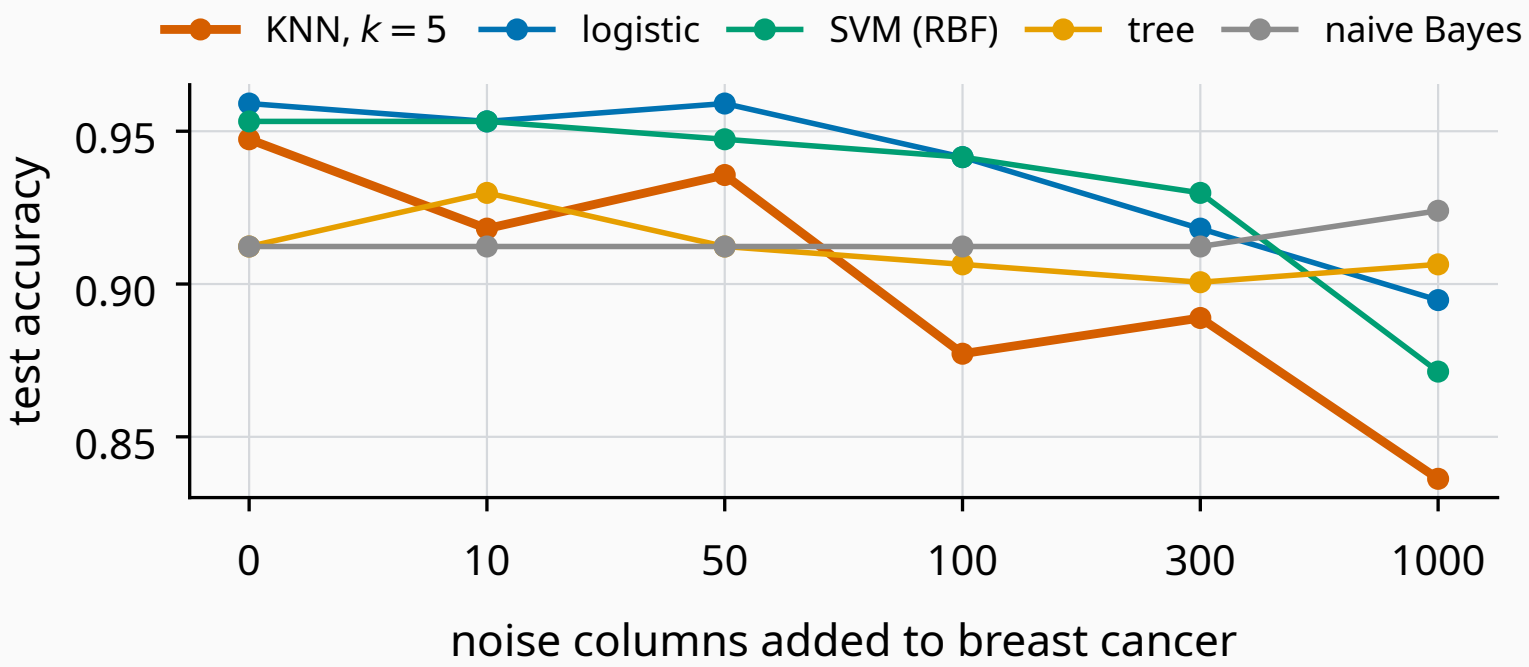
In high dimension *everything is far from everything*: 500 random points, one query.



- At $d = 1000$ the nearest point is only 10% closer than the farthest
- To hold 1% of the data at $d = 100$: edge **0.95**

Irrelevant Features

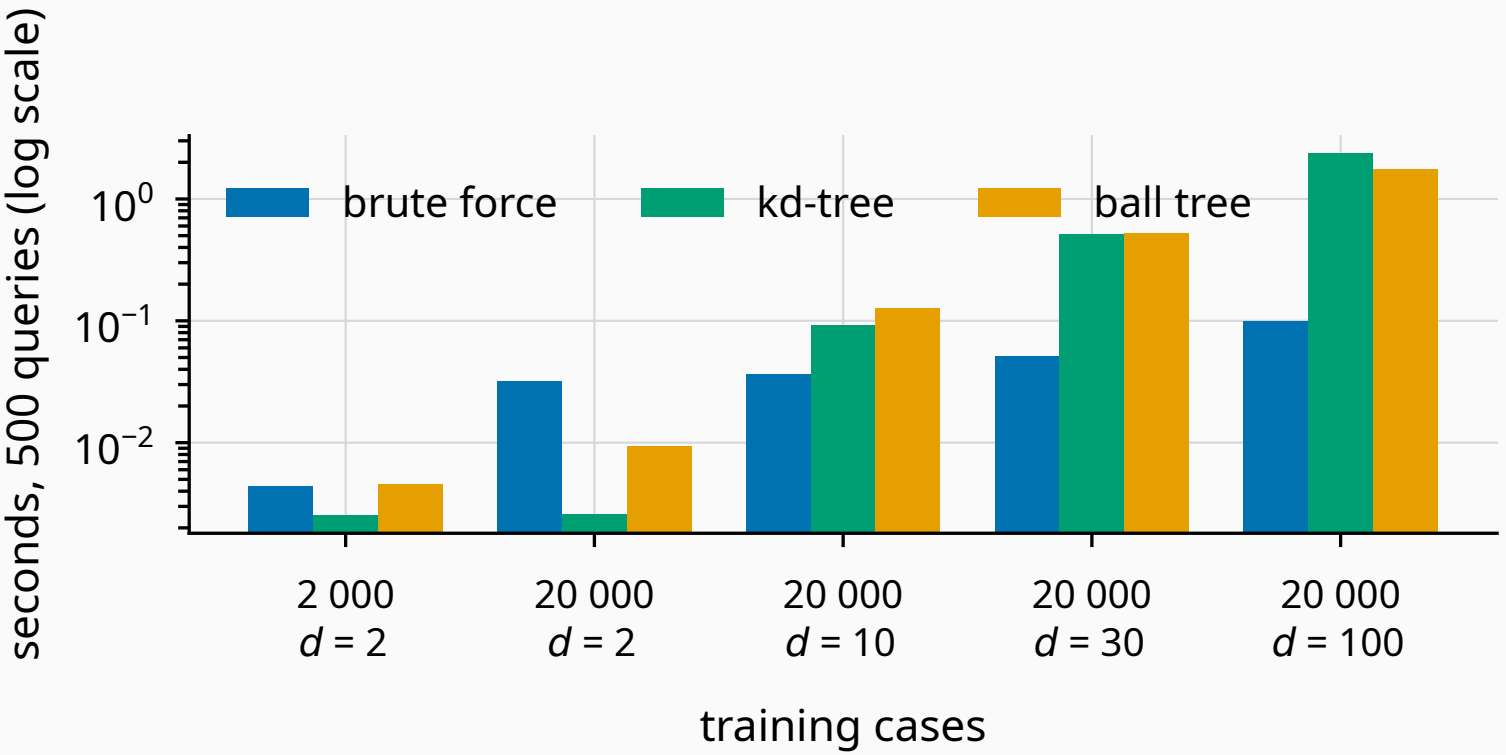
Add r **random** columns to the breast-cancer data. Test accuracy:



- Neighbours count every column: they degrade fastest
- A **tree** ignores a column that never gives a gain
- Remedy: select features *before* neighbours

The Cost of Prediction

Seconds to predict 500 queries, $k = 5$, by search method:

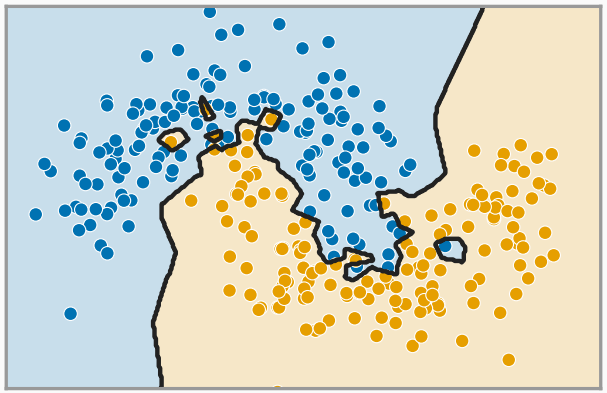


- A **kd-tree** prunes distances in low dimension; beyond $d \approx 10$ brute force wins
- Memory: 10^6 cases $\times$ 100 features = 0.8 gigabytes

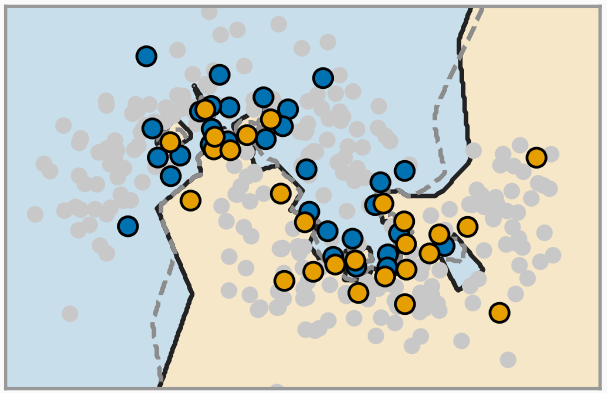
Prune the Data: Keep the Cases That Matter

Many stored cases never change a prediction: the ones near the class border define it. Two moons, 300 cases:

all 300 cases



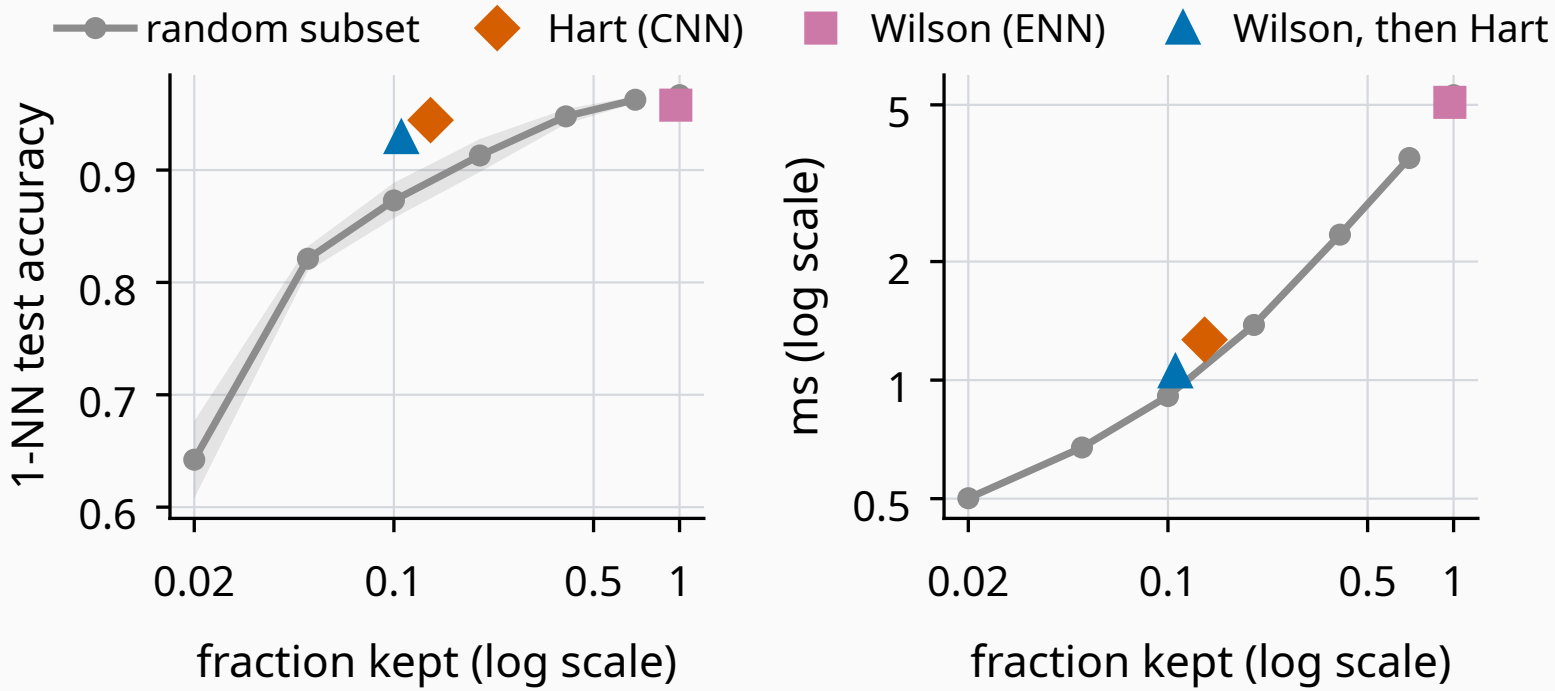
57 kept cases (large), 243 removed (grey)



- **Condensed** (Hart 1968): keep a case the kept ones misclassify
- 57 of 300 kept: test error 0.114, with all cases 0.091
- **Edited** (Wilson 1972): delete cases their neighbours outvote

Pruning in Numbers

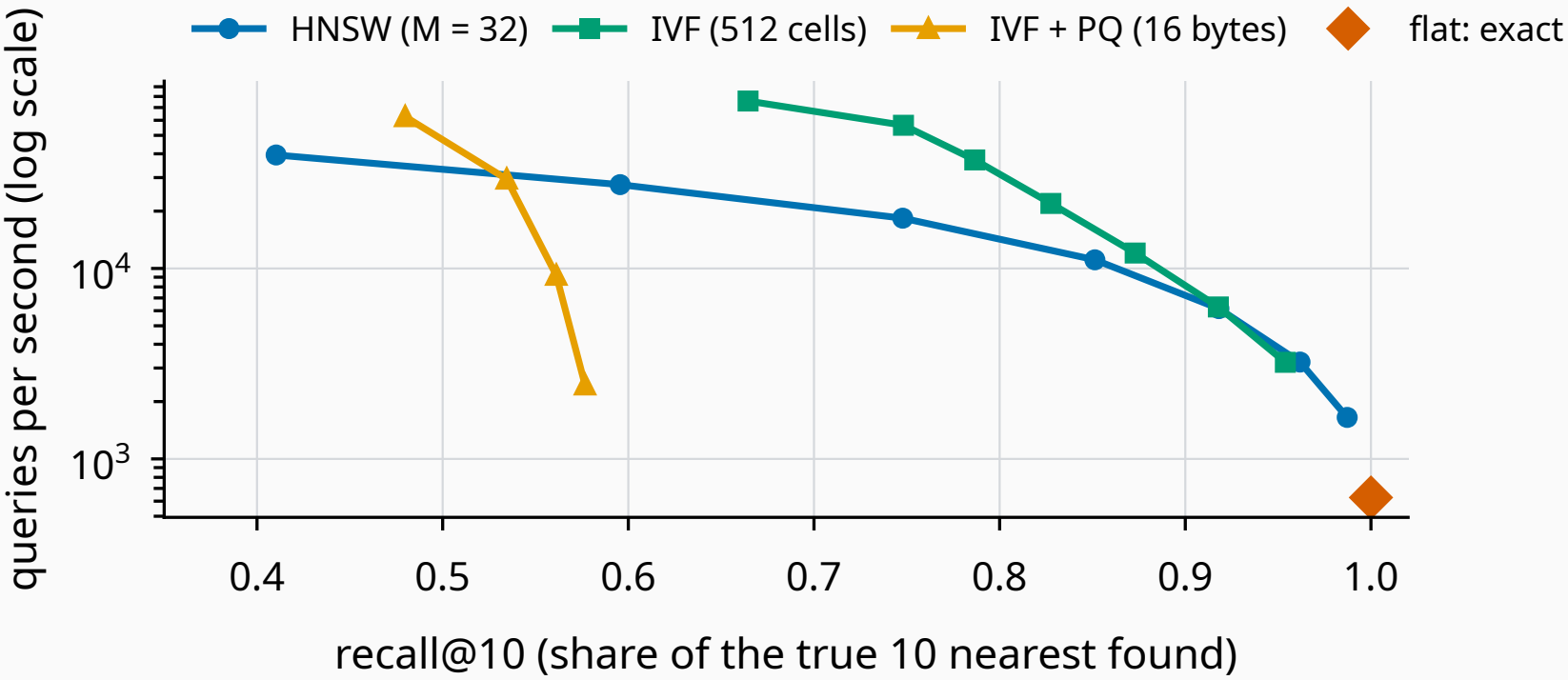
Digits, one neighbour, brute force; random subsets of the same size are the baseline.



- Hart keeps 0.134: accuracy 0.944 against 0.967, 5 times faster
- Noisy wine keeps 0.433: 0.759 against 0.795

Approximate Nearest Neighbours

Approximate search gives up the exact answer for speed.
Recall@10 is the share of the true 10 neighbours found.



- 200 000 vectors, $d = 64$: exact, 600 queries/s
- Cell index, 16 of 512 cells: recall 0.873, 20 times faster
- Graph index: 0.918, 10 times faster

How ANN Indexes Skip Distances

IVF
cluster the data into cells; search only the cells nearest to the query

HNSW
a graph of neighbours in layers; walk it towards the query

PQ
compress each vector to a few bytes; compare the codes

- **Faiss** (Meta): cell, graph and quantized indexes, on GPUs (Johnson et al. 2019)
- **hnswlib** and **Annoy** (Spotify); **ScaNN** (Google); graphs (Malkov and Yashunin 2018)
- **Vector databases** (Milvus, Qdrant, pgvector) wrap one of these

Summary: Strengths and Weaknesses of KNN

strengths	weaknesses
no training; adds new data at once	slow, memory-hungry prediction
non-linear boundary, any shape	needs scaling; **sensitive to irrelevant features**
few hyperparameters (k, distance)	suffers in high dimension
easy to explain: “like these cases”	no model to read or to export
multi-class and regression for free	no extrapolation; noisy for small k

- A **baseline** to try early: a few lines, and a number to beat
- A distance also serves clustering (Class 10), anomaly detection, recommendation

DEMO K1–K6: Notebook

`02_knn.ipynb` (K1 to K6) **shows** scaling, k, the curse of dimensionality, pruning, Faiss

Look for: one change at a time, on several data sets

Choosing a Learner

Recap: the Five Classes of Learners

class of learner	learns by	taught	what we built
Symbolists	rules, inverse deduction	06	decision tree
Connectionists	adjusting weights	03, 05	logistic regression, MLP
Evolutionaries	evolving candidates	02	genetic algorithm, DE
Bayesians	updating probabilities	04	naive Bayes, MAP
Analogizers	similarity to known cases	05, 06	SVM with a kernel, KNN

- Project 1: one learner **per class**, the same data and methodology (Domingos 2015)
- Each class has its own idea of *what is learned*

One Protocol, Five Learners (Demo K5)

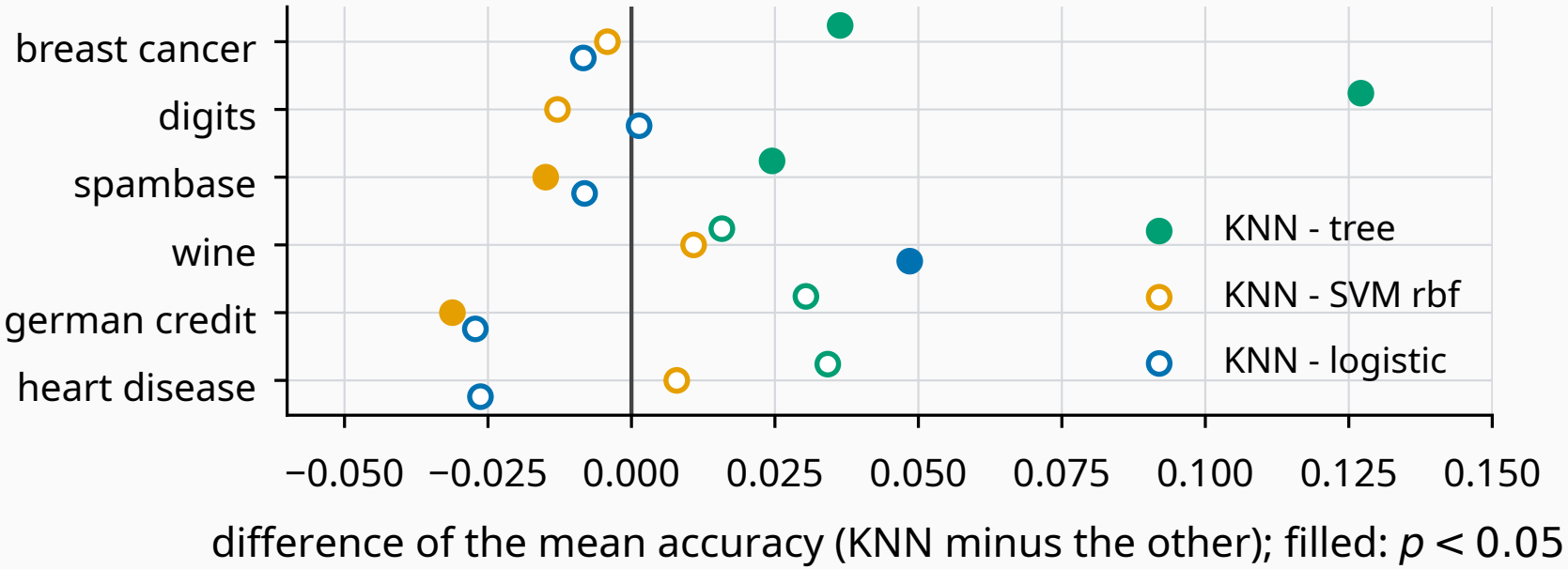
Median test accuracy of 5 stratified splits; k and `min_samples_leaf` tuned by inner cross-validation:

data set	logistic	naive Bayes	SVM (RBF)	tree	KNN
breast cancer	**0.972**	0.923	0.965	0.923	0.965
digits	0.969	0.816	**0.982**	0.842	0.967
spam	0.942	0.842	**0.950**	0.912	0.932
wine (≥ 6)	0.730	0.677	0.774	0.764	**0.780**
German credit	0.744	0.696	**0.752**	0.692	0.732
heart disease	**0.842**	**0.842**	**0.842**	0.776	0.829

- The support vector machine wins or ties on four of six
- Neighbours win on wine; standardization uses the training part

Is a Difference Real?

The corrected resampled t-test of Class 05: neighbours against three learners; filled dots have $p < 0.05$.



- Neighbours clearly **beat a single tree** on four of five sets
- Against the support vector machine and logistic regression the gaps are small

Summary: a Decision Guide

situation	first try
small tabular data, a baseline	logistic regression, KNN
rules must be explained to a person	decision tree (shallow)
non-linear, medium data (up to $\sim 10^4$)	SVM with RBF, KNN
many cases, high accuracy needed	network, ensembles (next class)
features on very different scales	any model, **after scaling** (not trees)
many irrelevant features	tree, regularized linear model, not KNN
must extrapolate	linear model, not tree or KNN

- Always: a **baseline**, a matching **split**, a **spread**, a **test**

Key Takeaways: Models and Limits

- A tree and nearest neighbours are **sets of cells**; any region with enough cells
- The price is **data and dimension**, not a functional form
- **Non-parametric** on the data: the model grows; hyperparameters remain
- The best tree is **NP-complete** (Hyafil and Rivest 1976): **greedy**, with exclusive or as its failure
- A category of v levels offers $2^{v-1} - 1$ questions
- Neighbours need a distance: one-hot or Gower, not codes

Key Takeaways: Trees

- **Tree:** greedy questions that maximize the **gain** (Gini, entropy)
- Numeric cuts at midpoints, found by one sorted scan
- An unlimited tree memorizes: limit the depth, or **prune**, on validation data
- **Regression tree:** leaf means, a staircase that **cannot extrapolate**
- **Strength:** readable rules; **weakness:** high variance, biased importances

Key Takeaways: KNN

- **Nearest neighbours:** no training; the k nearest cases vote or average
- The **distance** and k define the learner; **scale** the features
- **Curse of dimensionality:** noise features and high d destroy the distance
- **Prune** the cases or **approximate** the search to be faster
- Both are **local** and do not extrapolate; both are good baselines
- **Choose** on one protocol, with a spread and a test

Lab 06: Build It from Scratch

Part	Task	twin
A	Data visualization, evaluation methodology	—
B	Gini, entropy, the best threshold	`DecisionTreeClassifier`
C	A classification tree from scratch	`DecisionTreeClassifier`
D	Depth on validation data, pruning	`ccp_alpha`
E	A regression tree: variance reduction	`DecisionTreeRegressor`
F	KNN with `jax.numpy`	`KNeighborsClassifier`
G	KNN for regression	`KNeighborsRegressor`
H	Challenge: symbolist and analogizer	`GridSearchCV`

- **In class:** Parts A to C (60 minutes); **at home:** Parts D to H
- Guide `practice/06_trees_knn.pdf`; notebook `lab06_trees_knn.ipynb`
- **Project 1:** the tree is the *symbolist*; neighbours are the *analogizer*

References

Bentley, Jon Louis. 1975. “Multidimensional Binary Search Trees Used for Associative Searching.” Communications of the ACM 18 (9): 509–17.

Beyer, Kevin, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. 1999. “When Is ‘Nearest Neighbor’ Meaningful?” International Conference on Database Theory (ICDT), 217–35.

Breiman, Leo, Jerome Friedman, Richard Olshen, and Charles Stone. 1984. Classification and Regression Trees. Wadsworth.

Cover, Thomas, and Peter Hart. 1967. “Nearest Neighbor Pattern Classification.” IEEE Transactions on Information Theory 13 (1): 21–27.

Domingos, Pedro. 2015. The Master Algorithm. Basic Books.

Hart, Peter. 1968. “The Condensed Nearest Neighbor Rule.” IEEE Transactions on Information Theory 14 (3): 515–16.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. The Elements of Statistical Learning. 2nd ed. Springer.

Hyafil, Laurent, and Ronald L. Rivest. 1976. “Constructing Optimal Binary Decision Trees Is NP-Complete.” Information Processing Letters 5 (1): 15–17.

James, Gareth, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. 2023. An Introduction to Statistical Learning with Applications in Python. Springer.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2019. “Billion-Scale Similarity Search with GPUs.” IEEE Transactions on Big Data 7 (3): 535–47.

Malkov, Yu. A., and D. A. Yashunin. 2018. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.” IEEE Transactions on Pattern Analysis and Machine Intelligence 42 (4): 824–36.

Quinlan, J. Ross. 1986. “Induction of Decision Trees.” Machine Learning 1 (1): 81–106.

Stone, Charles J. 1977. “Consistent Nonparametric Regression.” The Annals of Statistics 5 (4): 595–620.

Strobl, Carolin, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. 2007. “Bias in Random Forest Variable Importance Measures.” BMC Bioinformatics 8: 25.

Wilson, Dennis L. 1972. “Asymptotic Properties of Nearest Neighbor Rules Using Edited Data.” IEEE Transactions on Systems, Man, and Cybernetics SMC-2 (3): 408–21.